

```
import sqlite3
import logging
import random
import time
import json
from datetime import
datetime, timedelta

from telegram import (
    Update,
    InlineKeyboardButton,
    InlineKeyboardMarkup,
    LabeledPrice,
)
from telegram.constants
import ParseMode
from telegram.ext import (
    Application,
    CommandHandler,
    CallbackQueryHandler,
```

```
        MessageHandler ,
        PreCheckoutQueryHandler ,
        ContextTypes ,
        filters ,
    )

    logging.basicConfig(
        format="%(asctime)s - %(
name)s - %(levelname)s - %(
message)s",
        level=logging.INFO,
    )
    logger =
logging.getLogger("LIBER")

#
=====
=====
=====
# تنظیمات کلی
#
=====
```

```
=====
=====
```

```
BOT_TOKEN =
"YOUR_BOT_TOKEN_HERE"
ADMIN_IDS = [123456789]
```

```
FORCE_JOIN_CHANNELS = [
    {"id": "@Libercoin1",
    "title": "کانال LIBER", "url":
    "https://t.me/Libercoin1"},
    # کانال یا گروه دیگری داری، همینجا #
    # یک آیتم دیگر اضافه کن:
    # {"id":
    "@your_channel_2", "title":
    "کانال دوم", "url":
    "https://t.me/your_channel_2"},
    ]
```

```
DB_PATH = "liber.db"
```

```
MARKET_BASE_PRICE = 100
BUY_FEE_PERCENT = 2
SELL_FEE_PERCENT = 2
MARKET_FLUCTUATION_RANGE =
(-0.07, 0.07)    # هر ساعت
MARKET_UPDATE_INTERVAL_SECONDS = 3600    # هر ۱ ساعت
```

```
BANK_INTEREST_PERCENT =
1.5    # سود روزانه سپرده
LOAN_INTEREST_PERCENT =
5    # کارمزد وام
MAX_LOAN_MULTIPLIER =
3    # سقف وام بر اساس
سطح
```

```
XP_PER_LEVEL = 100
DAILY_MISSION_XP = 20
DAILY_MISSION_LIBER = 8    #
سخت‌تر شد (قبلاً ۱۵)
```

```
CHEST_TABLE = {
```

```
    "free":      {"cost":
    {}, "rewards": [("coin", 40,
    120), ("liber", 1, 3)]},
    "bronze":    {"cost":
    {"coin": 350}, "rewards":
    [("coin", 90, 350),
    ("liber", 2, 7), ("xp", 8,
    18)]},
    "silver":    {"cost":
    {"coin": 900}, "rewards":
    [("liber", 6, 20),
    ("diamond", 1, 2), ("xp",
    15, 35)]},
    "gold":      {"cost":
    {"liber": 130}, "rewards":
    [("liber", 18, 55),
    ("diamond", 2, 4), ("medal",
    1, 2)]},
    "diamond":   {"cost":
    {"diamond": 25}, "rewards":
    [("liber", 50, 140),
    ("diamond", 4, 8), ("medal",
```

```
1, 4)]]},  
}
```

```
VIP_TIERS = {  
    "silver":  
    {"cost_diamond": 50,  
     "xp_bonus": 1.1,  
     "income_bonus": 1.1},  
    "gold":  
    {"cost_diamond": 150,  
     "xp_bonus": 1.25,  
     "income_bonus": 1.25},  
    "diamond":  
    {"cost_diamond": 400,  
     "xp_bonus": 1.5,  
     "income_bonus": 1.5},  
    "titan":  
    {"cost_diamond": 1000,  
     "xp_bonus": 2.0,  
     "income_bonus": 2.0},  
}
```

```
# -----  
-----  
-----  
# اشتراک ویژه با تلگرام استارز  
(Telegram Stars) ★  
# -----  
-----  
-----  
# است (XTR) قیمت‌ها به ستاره تلگرام  
- قیمت‌ها منصفانه و شفاف نگه داشته  
شده‌اند.  
STAR_SUBSCRIPTIONS = {  
    "normal": {  
        "title": "🏆 اشتراک  
عادی",  
        "benefits": "درآمد +۱۰٪  
| یک صندوق نقره‌ای رایگان | XP +۱۰٪  
در ماه",  
        "durations": {30:  
روز: قیمت به { # {  
استارز  
},
```

```
"dragon": {
  "title": "🐉 اشتراک
دراگون",
  "benefits": "۲۵% درآمد
یک صندوق طلایی رایگان | XP +۲۵%
",
  "durations": {30:
150, 90: 380},
},
"dragon_legend": {
  "title": "🐉 اشتراک
دراگون لجند",
  "benefits": "۵۰% درآمد
صندوق الماسی رایگان در | XP +۵۰%
ماه | قاب اختصاصی لجند | ورود رایگان
",
  "durations": {30:
300, 90: 750},
},
}
```

```
-----  
-----  
# با تلگرام استارز (نرخ LIBER خرید  
منصفانه و ثابت)  
# -----  
-----  
-----  
STAR_LIBER_PACKS = {  
    "pack_small": {"title":  
        "📦 بسته کوچک", "liber": 100,  
        "stars": 50},  
    "pack_medium": {"title":  
        "📦 بسته متوسط", "liber":  
        300, "stars": 130},  
    "pack_large": {"title":  
        "📦 بسته بزرگ", "liber": 1000,  
        "stars": 400},  
    "pack_mega": {"title":  
        "📦 بسته مگا", "liber": 3000,  
        "stars": 1100},  
}
```

```
# -----  
-----  
-----  
#   کد هدیه (Gift Code)  
# -----  
-----  
-----  
GIFT_CODE_MAX_USES_DEFAULT =  
1  
  
# -----  
-----  
-----  
#   برداشت TON  
# -----  
-----  
-----  
MIN_WITHDRAW_LIBER = 2000  
WITHDRAW_FEE_PERCENT = 5    #  
کارمزد برداشت  
  
LEAGUE_THRESHOLDS = [
```

```
(0, "🥉 برنز"),  
(500, "🥈 نقره"),  
(1500, "🥇 طلا"),  
(4000, "💎 پلاتینیوم"),  
(10000, "💎 الماس"),  
(25000, "👑 تایتان"),  
(60000, "🌌 افسانه‌ای"),  
]
```

```
SPAM_COOLDOWN_SECONDS = 1.0  
_last_action_time = {}  
_warn_count = {}
```

```
# -----  
-----  
-----  
# سیستم هوشمند ضد تقلب  
# -----  
-----  
-----  
CHEAT_ACTION_WINDOW_SECONDS  
= 10      # بازه زمانی بررسی
```

```
CHEAT_ACTION_MAX_IN_WINDOW =  
12      # بیش از این تعداد کلیک در  
بازه = مشکوک  
CHEAT_FLAG_BAN_THRESHOLD =  
4        # بعد از این تعداد پرچم  
مشکوک، خودکار مسدود می‌شود  
SUSPICIOUS_LIBER_GAIN_THRESH  
OLD = 2000 # افزایش ناگهانی بیش  
از این مقدار در یک عملیات، بررسی  
می‌شود
```

```
_action_log = {}      #  
user_id -> [timestamps]  
_cheat_flags = {}     #  
user_id -> count
```

```
# -----  
-----  
-----  
# فروشگاه  
# -----  
-----
```

```
SHOP_ITEMS = {  
    "energy_50": {"title":  
        "⚡ ۵۰ انرژى", "cost":  
        {"coin": 200}, "give":  
        ("energy", 50)},  
    "energy_200": {"title":  
        "⚡ ۲۰۰ انرژى", "cost":  
        {"coin": 700}, "give":  
        ("energy", 200)},  
    "diamond_10": {"title":  
        "💎 ۱۰ الماس", "cost":  
        {"liber": 150}, "give":  
        ("diamond", 10)},  
    "diamond_50": {"title":  
        "💎 ۵۰ الماس", "cost":  
        {"liber": 650}, "give":  
        ("diamond", 50)},  
    "frame_gold": {"title":  
        "🖼️ قاب طلايى", "cost":  
        {"diamond": 30}, "give":  
        ("frame", "gold")},
```

```
    "frame_neon": {"title":  
    "🖼️ قاب نئونی", "cost":  
    {"diamond": 60}, "give":  
    ("frame", "neon")},  
    }
```

```
# -----  
-----  
-----
```

```
# دستاوردها
```

```
# -----  
-----  
-----
```

```
ACHIEVEMENTS = {  
    "first_trade":  
    {"title": "🏆 اولین معامله",  
    "desc": "اولین خرید یا فروش در  
    بازار", "reward_liber": 20},  
    "trader_100":  
    {"title": "📈  
    معامله‌گر", "desc": "۱۰۰  
    بار در بازار معامله کن",
```

```
"reward_liber": 200},
  "chest_opener":
{"title": "📦",
"desc": "۵۰ صندوق باز",
"reward_diamond": 20},
  "country_founder":
{"title": "🏛️",
"desc": "یک بنیان‌گذار",
"reward_coin": 300},
  "level_10":
{"title": "⭐ سطح ۱۰",
"desc": "به سطح ۱۰ برس",
"reward_diamond": 30},
  "level_25":
{"title": "🌟 سطح ۲۵",
"desc": "به سطح ۲۵ برس",
"reward_diamond": 100},
  "referral_10":
```

```
{
  "title": "👥",
  "desc": "۱۰",
  "reward_liber": 300,
  "alliance_join": {
    "title": "🤝",
    "desc": "به",
    "reward_coin": 200,
    "vip_member": {
      "title": "👑 عضو",
      "desc": "هر",
      "reward_medal": 5,
      "bank_saver": {
        "title": "🏦 پس انداز",
        "desc": "۱۰۰۰ Coin",
        "reward_coin": 150
      }
    }
  }
}
```

```
-----  
-----  
# تحقیقات / فناوری  
# -----  
-----  
-----  
RESEARCH_TREE = [  
    {"level": 1, "name":  
"کشاورزی مدرن", "cost_coin":  
300, "effect": "production  
+10%"},  
    {"level": 2, "name":  
"معدن کاری پیشرفته", "cost_coin":  
700, "effect": "production  
+20%"},  
    {"level": 3, "name":  
"انرژی خورشیدی", "cost_coin":  
1500, "effect": "production  
+35%"},  
    {"level": 4, "name":  
"هوش مصنوعی صنعتی",  
"cost_coin": 3000, "effect":
```

```
"production +50%"},
    {"level": 5, "name":
"فناوری کوانتومی", "cost_coin":
6000, "effect": "production
+75%"},
]
```

```
# -----
-----
-----
```

```
# دفاع نظامی
```

```
# -----
-----
-----
```

```
DEFENSE_UPGRADE_BASE_COST =
250
```

```
DEFENSE_UPGRADE_GROWTH = 1.6
```

```
# -----
-----
-----
```

```
# اکتشاف
```

```
# -----  
-----  
  
EXPLORATION_MIN_LEVEL = 5  
EXPLORATION_ENERGY_COST = 20  
EXPLORATION_REWARDS = [  
    ("coin", 50, 300),  
    ("liber", 5, 40),  
    ("diamond", 0, 3),  
]  
  
# -----  
-----  
  
# بازار سیاه (روزانه تغییر می‌کند)  
# -----  
-----  
  
BLACK_MARKET_POOL = [  
    {"title": "👑 آیتم افسانه‌ای",  
     "cost": {"diamond":  
80}, "give": ("medal", 10)},
```

```
        {"title": "💎 ویژه پیشنهاد",
         "cost": {"liber": 500}, "give": ("diamond", 40)},
        {"title": "📦 جعبه رمز و راز",
         "cost": {"coin": 1000}, "give": ("liber", 60)},
        {"title": "🏅 مدال کمیاب",
         "cost": {"diamond": 40}, "give": ("medal", 5)},
    ]
```

```
# -----
# -----
# -----
# فصل بازی
# -----
# -----
SEASON_LENGTH_DAYS = 90
```

```
# -----  
-----  
-----  
# معامله مستقیم بین بازیکنان  
# -----  
-----  
-----  
TRADE_FEE_PERCENT = 3  
  
# -----  
-----  
-----  
# بازار پیش‌بینی قیمت  
# -----  
-----  
-----  
PREDICTION_BET_AMOUNT = 50  
PREDICTION_WIN_MULTIPLIER =  
1.8  
  
# -----
```

```
-----  
-----  
# فیلتر فحش (نمونه ساده - قابل  
گسترش)  
# -----  
-----  
-----  
BANNED_WORDS = ["kosekhar",  
"fuckyou"]  
MAX_WARN_BEFORE_BAN = 5  
  
# -----  
-----  
-----  
# رقابت آنلاین (فوتبال / بسکتبال)  
# -----  
-----  
-----  
SPORTS = {  
    "football": {  
        "title": "🏈 فوتبال",  
        "stats": {
```

```
    "life": "❤️ جون",
    "accuracy": "🎯",
    "intensity": "🔥",
    "shot": "🏀",
    "technique": "🌀",
    "physical": "💪",
  },
  "basketball": {
    "title": "🏀 بسکتبال",
    "stats": {
      "speed": "⚡",
      "accuracy": "🎯",
      "press": "🧱",
    },
  },
}
```

```
        "physical": "💪",
        "life": "❤️ جون",
    },
    },
}
```

```
STAT_UPGRADE_BASE_COST = 40
STAT_UPGRADE_GROWTH = 1.42
# هرچه سطح بالاتر، ارتقا سخت‌تر و
پرهزینه‌تر می‌شود
STAT_MAX_LEVEL = 50
```

```
MATCH_ENTRY_FEE =
20 # هزینه ورود به هر
(LIBER) مسابقه رنک
MATCH_POT_FEE_PERCENT =
18 # کارمزد ربات از مجموع جایزه
(سخت‌تر شد، قبلاً ۱۲٪)
MATCH_POSSESSIONS =
5 # تعداد حمله در هر مسابقه
```



```
HARD_MATCH_ENTRY_FEE =  
4000      # هزینه ورود مسابقه سخت  
(Coin)  
HARD_MATCH_REWARD =  
8000      # جایزه برد مسابقه سخت  
(Coin)  
HARD_MATCH_OPPONENT_BOOST =  
1.35      # حریف سخت قوی‌تر از خودت  
شبیه‌سازی می‌شود
```

```
RANK_WIN_POINTS = 15  
RANK_DRAW_POINTS = 6  
RANK_LOSS_POINTS = -5
```

```
LEAGUE_TIERS = [  
    (0,      "🥉 مبتدی"),  
    (100,    "🥈 حرفه‌ای"),  
    (300,    "🥇 استاد"),  
    (700,    "🐉 اژدهای آزاد"),  
    (1500,   "🐉 اژدهای  
    افسانه‌ای"),  
    (3000,   "🐉👑 اژدهای کامل")
```

```
), "افسانه‌ای",  
    (6000, "💎 لیبر لجند وان"),  
]
```

```
TOURNAMENT_LENGTH_DAYS =  
60      # هر ۲ ماه  
TOURNAMENT_REWARDS = {1:  
700, 2: 500, 3: 300}    #  
LIBER (نفرات اول تا سوم)  
TOURNAMENT_MEDAL_REWARDS =  
{4: 100, 5: 50}        # مدال  
(نفرات چهارم و پنجم)
```

```
#  
=====
```

```
# دیتابیس
```

```
#  
=====
```

```
====
```

```
def db():  
    conn =  
    sqlite3.connect(DB_PATH)  
    conn.row_factory =  
    sqlite3.Row  
    return conn
```

```
def init_db():  
    conn = db()  
    c = conn.cursor()  
    c.execute(  
        """  
        CREATE TABLE IF NOT  
EXISTS users (  
            user_id INTEGER  
PRIMARY KEY,  
            username TEXT,  
            first_name TEXT,  
            joined_at TEXT,
```

```
        last_seen TEXT,  
        login_count  
INTEGER DEFAULT 1,  
        level INTEGER  
DEFAULT 1,  
        xp INTEGER  
DEFAULT 0,  
        liber REAL  
DEFAULT 100,  
        coin REAL  
DEFAULT 500,  
        energy INTEGER  
DEFAULT 100,  
        diamond INTEGER  
DEFAULT 0,  
        medal INTEGER  
DEFAULT 0,  
        vip TEXT DEFAULT  
'none',  
        country_name  
TEXT DEFAULT '',  
        country_pop
```

```
INTEGER DEFAULT 0,  
        country_budget  
REAL DEFAULT 0,  
        bank_deposit  
REAL DEFAULT 0,  
        loan_amount REAL  
DEFAULT 0,  
        alliance_id  
INTEGER DEFAULT 0,  
        ref_by INTEGER  
DEFAULT 0,  
        ref_count  
INTEGER DEFAULT 0,  
  
last_daily_mission TEXT  
DEFAULT '',  
  
last_daily_reward TEXT  
DEFAULT '',  
        banned INTEGER  
DEFAULT 0,  
        warn_count
```

```
INTEGER DEFAULT 0,  
        frame TEXT  
DEFAULT 'normal',  
        trade_count  
INTEGER DEFAULT 0,  
        chest_count  
INTEGER DEFAULT 0,  
        research_level  
INTEGER DEFAULT 0,  
        defense_level  
INTEGER DEFAULT 0,  
        achievements  
TEXT DEFAULT '[]',  
        bio TEXT DEFAULT  
'',  
        football_life  
INTEGER DEFAULT 10,  
  
football_accuracy INTEGER  
DEFAULT 10,  
  
football_intensity INTEGER
```

```
DEFAULT 10,  
        football_shot  
INTEGER DEFAULT 10,  
  
football_technique INTEGER  
DEFAULT 10,  
  
football_physical INTEGER  
DEFAULT 10,  
        basketball_speed  
INTEGER DEFAULT 10,  
  
basketball_accuracy INTEGER  
DEFAULT 10,  
        basketball_press  
INTEGER DEFAULT 10,  
  
basketball_physical INTEGER  
DEFAULT 10,  
        basketball_life  
INTEGER DEFAULT 10,  
        rank_points
```

```

INTEGER DEFAULT 0,
            matches_played
INTEGER DEFAULT 0,
            matches_won
INTEGER DEFAULT 0,
            vip_expires_at
TEXT DEFAULT ''
        )
    """

)
c.execute(
    """

        CREATE TABLE IF NOT
EXISTS market (
            id INTEGER
PRIMARY KEY,
            price REAL
        )
    """

)
c.execute(
    """

```



```

        CREATE TABLE IF NOT
        EXISTS alliances (
            alliance_id
        INTEGER PRIMARY KEY
        AUTOINCREMENT,
            name TEXT,
            leader_id
        INTEGER,
            treasury REAL
        DEFAULT 0
        )
        """
    )
    c.execute(
        """
        CREATE TABLE IF NOT
        EXISTS trades (
            trade_id INTEGER
        PRIMARY KEY AUTOINCREMENT,
            seller_id
        INTEGER,
            item_field TEXT,

```

```

        item_amount
REAL,
        price_coin REAL,
        status TEXT
DEFAULT 'open',
        buyer_id INTEGER
DEFAULT 0,
        created_at TEXT
    )
    """
)
c.execute(
    """
        CREATE TABLE IF NOT
EXISTS predictions (
            pred_id INTEGER
PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER,
            direction TEXT,
            start_price
REAL,
            bet_amount REAL,

```

```
        status TEXT
DEFAULT 'open',
        created_at TEXT
    )
    """
)
c.execute(
    """
        CREATE TABLE IF NOT
EXISTS season (
            id INTEGER
PRIMARY KEY,
            season_number
INTEGER,
            started_at TEXT
        )
        """
)
c.execute(
    """
        CREATE TABLE IF NOT
EXISTS black_market_stock (
```

```

        id INTEGER
PRIMARY KEY,
        item_index
INTEGER,
        day TEXT
    )
    """
)
c.execute(
    """
        CREATE TABLE IF NOT
EXISTS match_queue (
            user_id INTEGER
PRIMARY KEY,
            sport TEXT,
            joined_at TEXT
        )
    """
)
c.execute(
    """
        CREATE TABLE IF NOT

```

```

EXISTS matches (
            match_id INTEGER
PRIMARY KEY AUTOINCREMENT,
            player_id
INTEGER,
            opponent_id
INTEGER,
            sport TEXT,
            player_score
INTEGER,
            opponent_score
INTEGER,
            result TEXT,
            log TEXT,
            created_at TEXT
        )
    """
)
c.execute(
    """
    CREATE TABLE IF NOT
EXISTS tournament (

```

```
        id INTEGER
PRIMARY KEY,
        started_at TEXT
    )
    """
)
c.execute(
    """
    CREATE TABLE IF NOT
EXISTS gift_codes (
        code TEXT
PRIMARY KEY,
        reward_field
TEXT,
        reward_amount
REAL,
        max_uses
INTEGER,
        used_count
INTEGER DEFAULT 0,
        created_by
INTEGER,
```

```
        created_at TEXT
    )
    """
)
c.execute(
    """
    CREATE TABLE IF NOT
    EXISTS gift_code_redemptions
    (
        code TEXT,
        user_id INTEGER,
        redeemed_at
    TEXT,
        PRIMARY KEY
    (code, user_id)
    )
    """
)
c.execute(
    """
    CREATE TABLE IF NOT
    EXISTS withdrawal_requests (
```

```

        request_id
INTEGER PRIMARY KEY
AUTOINCREMENT,
        user_id INTEGER,
        amount_liber
REAL,
        fee_liber REAL,
        ton_address
TEXT,
        status TEXT
DEFAULT 'pending',
        created_at TEXT,
        processed_at
TEXT
    )
    """
)
c.execute("SELECT
COUNT(*) as cnt FROM
tournament")
    if c.fetchone()["cnt"]
== 0:

```



```
        c.execute(
            "INSERT INTO
tournament (id, started_at)
VALUES (1, ?)",

(datetime.now().strftime("%Y
-%m-%d"),),
        )
        c.execute("SELECT
COUNT(*) as cnt FROM
market")
        if c.fetchone()["cnt"]
== 0:
            c.execute("INSERT
INTO market (id, price)
VALUES (1, ?)",
(MARKET_BASE_PRICE,))
            c.execute("SELECT
COUNT(*) as cnt FROM
season")
            if c.fetchone()["cnt"]
== 0:
```

```
        c.execute(
            "INSERT INTO
season (id, season_number,
started_at) VALUES (1, 1,
?)" ,

(datetime.now().strftime("%Y
-%m-%d"),),
        )
        conn.commit()
        conn.close()

def get_user(user_id):
    conn = db()
    c = conn.cursor()
    c.execute("SELECT * FROM
users WHERE user_id=?",
(user_id,))
    row = c.fetchone()
    conn.close()
    return row
```

```
def set_field(user_id,
field, value):
    conn = db()
    c = conn.cursor()
    c.execute(f"UPDATE users
SET {field}=? WHERE
user_id=?", (value,
user_id))
    conn.commit()
    conn.close()
```

```
def add_currency(user_id,
field, amount):
    conn = db()
    c = conn.cursor()
    c.execute(f"UPDATE users
SET {field} = {field} + ?
WHERE user_id=?", (amount,
user_id))
```

```
conn.commit()
conn.close()

def add_xp(user_id, amount):
    u = get_user(user_id)
    vip = u["vip"]
    bonus =
VIP_TIERS.get(vip,
{}).get("xp_bonus", 1.0)
    amount = int(amount *
bonus)
    new_xp = u["xp"] +
amount
    new_level = u["level"]
    while new_xp >=
new_level * XP_PER_LEVEL:
        new_xp -= new_level
* XP_PER_LEVEL
        new_level += 1
    conn = db()
    c = conn.cursor()
```

```
        c.execute("UPDATE users
SET xp=?, level=? WHERE
user_id=?", (new_xp,
new_level, user_id))
        conn.commit()
        conn.close()
        return new_level
```

```
def get_market_price():
    conn = db()
    c = conn.cursor()
    c.execute("SELECT price
FROM market WHERE id=1")
    price = c.fetchone()
    ["price"]
    conn.close()
    return price
```

```
def set_market_price(price):
    conn = db()
```

```
        c = conn.cursor()
        c.execute("UPDATE market
SET price=? WHERE id=1",
(price,))
        conn.commit()
        conn.close()
```

```
def
get_league_name(xp_total):
    name =
LEAGUE_THRESHOLDS[0][1]
    for threshold,
league_name in
LEAGUE_THRESHOLDS:
        if xp_total >=
threshold:
            name =
league_name
    return name
```

```

def
create_user_if_not_exists(tg
_user, ref_by=0):
    conn = db()
    c = conn.cursor()
    c.execute("SELECT
user_id FROM users WHERE
user_id=?", (tg_user.id,))
    existing = c.fetchone()
    now =
datetime.now().strftime("%Y-
%m-%d %H:%M:%S")
    if existing:
        c.execute(
            "UPDATE users
SET last_seen=?,
login_count=login_count+1,
username=?, first_name=?
WHERE user_id=?",
            (now,
tg_user.username or "",
tg_user.first_name or "",

```

```
tg_user.id),
    )
    conn.commit()
    conn.close()
    return False
else:
    c.execute(
        """
            INSERT INTO
users (user_id, username,
first_name, joined_at,
last_seen, ref_by)
            VALUES (?, ?, ?,
?, ?, ?)
            """,
            (tg_user.id,
tg_user.username or "",
tg_user.first_name or "",
now, now, ref_by),
        )
    if ref_by:
        c.execute(
```



```
                "UPDATE
users SET
ref_count=ref_count+1,
liber=liber+50,
coin=coin+200 WHERE
user_id=?",
                (ref_by,),
            )
            conn.commit()
            conn.close()
            return True
```

```
def is_banned(user_id):
    u = get_user(user_id)
    return bool(u and
u["banned"] == 1)
```

```
def
anti_spam_check(user_id):
    now = time.time()
```

```
        last =
        _last_action_time.get(user_id, 0)
        if now - last <
        SPAM_COOLDOWN_SECONDS:
            _warn_count[user_id]
        = _warn_count.get(user_id,
        0) + 1
            return False,
            _warn_count[user_id]

        _last_action_time[user_id] =
        now
        _warn_count[user_id] = 0
        return True, 0
```

```
def
check_click_rate_cheat(user_
id):
    """با یک پنجره زمانی متحرک،
    الگوی کلیک غیرطبیعی (اسکرپت/بات) را
```

.تشخیص می‌دهد

اگر رفتار مشکوک True: خروجی

تشخیص داده شد

```
now = time.time()
```

```
log =
```

```
_action_log.setdefault(user_id, [])
```

```
log.append(now)
```

```
# فقط رویدادهای داخل بازه زمانی
```

را نگه می‌داریم

```
cutoff = now -
```

```
CHEAT_ACTION_WINDOW_SECONDS
```

```
while log and log[0] <
```

```
cutoff:
```

```
log.pop(0)
```

```
if len(log) >
```

```
CHEAT_ACTION_MAX_IN_WINDOW:
```

```
_cheat_flags[user_id] =
```

```
_cheat_flags.get(user_id, 0)
```

```
+ 1
```

```
log.clear()
return True
return False
```

```
async def
flag_suspicious_activity(use
r_id, context, reason):
    """یک پرچم تخلف برای کاربر ثبت
    می‌کند؛ در صورت عبور از حد مجاز،
    خودکار مسدود می‌شود."""
    count =
    _cheat_flags.get(user_id, 0)
    if count >=
    CHEAT_FLAG_BAN_THRESHOLD:
        set_field(user_id,
        "banned", 1)
        for admin_id in
        ADMIN_IDS:
            try:
                await
                context.bot.send_message(
```

```
admin_id,
```

f'' 

کاربر
تشخیص خودکار قلب

به دلیل `{user_id}`

«{reason}» "

و عبور از f

حد مجاز پرچم‌های مشکوک، به صورت

، "خودکار مسدود شد

```
parse_mode=ParseMode.HTML,
```

)

except

Exception:

pass

```
elif count > 0:
```

```
for admin_id in
```

ADMIN_IDS:

```
try:
```

await

```
context.bot.send_message(
```

```

admin_id,
                                f"⚠️
<b>کاربر</b> فعالیت مشکوک</b>\n\n
<code>{user_id}</code>:
{reason} "

                                f"پرچم (
{count}/{CHEAT_FLAG_BAN_THRESHOLD}) ",

parse_mode=ParseMode.HTML,
                                )
                                except
Exception:
                                pass

def
check_suspicious_liber_gain(
user_id, amount,
threshold=SUSPICIOUS_LIBER_GAIN_THRESHOLD):
    """ LIBER اگر مقدار یک تراکنش """

```

از حد مجاز بیشتر باشد، به عنوان مشکوک
علامت می زند.""

```
    return amount is not  
None and amount >= threshold
```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
# دستاوردها
```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
def
```

```
get_achievements(user_id):
```

```
    u = get_user(user_id)
```

```
    try:
```

```
        return
```

```
    json.loads(u["achievements"])
```

```
)  
    except Exception:  
        return []  
  
def  
unlock_achievement(user_id,  
key):  
    unlocked =  
get_achievements(user_id)  
    if key in unlocked:  
        return False  
    unlocked.append(key)  
    set_field(user_id,  
"achievements",  
json.dumps(unlocked))  
    ach = ACHIEVEMENTS[key]  
    if "reward_liber" in  
ach:  
  
add_currency(user_id,  
"liber",
```



```
ach["reward_liber"])
    if "reward_coin" in ach:

add_currency(user_id,
"coin", ach["reward_coin"])
    if "reward_diamond" in
ach:

add_currency(user_id,
"diamond",
ach["reward_diamond"])
    if "reward_medal" in
ach:

add_currency(user_id,
"medal",
ach["reward_medal"])
    return True

def
check_achievements(user_id):
```

بررسی و باز کردن خودکار
دستاوردهایی که شرایطشان برآورده شده.
خروجی: لیست عنوان‌های تازه باز
شده."

```
u = get_user(user_id)
newly_unlocked = []

checks = {
    "first_trade":
u["trade_count"] >= 1,
    "trader_100":
u["trade_count"] >= 100,
    "chest_opener":
u["chest_count"] >= 50,
    "country_founder":
bool(u["country_name"]),
    "level_10":
u["level"] >= 10,
    "level_25":
u["level"] >= 25,
    "referral_10":
u["ref_count"] >= 10,
```

```

        "alliance_join":
u["alliance_id"] != 0,
        "vip_member":
u["vip"] != "none",
        "bank_saver":
u["bank_deposit"] >= 1000,
    }
    for key, condition in
checks.items():
        if condition and
unlock_achievement(user_id,
key):

newly_unlocked.append(ACHIEV
EMENTS[key]["title"])
    return newly_unlocked

#
=====
=====
=====

```

```

# تحقیقات / فناوری
#
=====
=====
=====

def
get_research_info(user_id):
    u = get_user(user_id)
    level =
u["research_level"]
    if level >=
len(RESEARCH_TREE):
        return None
    return
RESEARCH_TREE[level]

def
upgrade_research(user_id):
    u = get_user(user_id)
    info =

```

```

get_research_info(user_id)
    if not info:
        return False, "تمام 📖
سطوح تحقیقاتی را کامل کرده‌ای."
        if u["coin"] <
info["cost_coin"]:
            return False, "❌
Coin کافی نداری."
            add_currency(user_id,
"coin", -info["cost_coin"])
            set_field(user_id,
"research_level",
u["research_level"] + 1)
            return True, f"تحقیق 📖
«{info['name']}» تکمیل شد
({info['effect']})"

#
=====
=====
=====

```

```

# دفاع نظامی
#
=====
=====
=====

def
get_defense_upgrade_cost(current_level):
    return
    round(DEFENSE_UPGRADE_BASE_COST *
    (DEFENSE_UPGRADE_GROWTH **
    current_level), 2)

def
upgrade_defense(user_id):
    u = get_user(user_id)
    cost =
    get_defense_upgrade_cost(u["
    defense_level"])

```

```
if u["coin"] < cost:
    return False, f"❌
نیاز {cost} Coin برای ارتقا به
داری."
    add_currency(user_id,
"coin", -cost)
    set_field(user_id,
"defense_level",
u["defense_level"] + 1)
    return True, f"🛡️ دفاع
کشورت به سطح
{u['defense_level']+1} ارتقا
یافت."
```

```
#
=====
=====
=====
# اکتشاف
#
=====
```

```
=====
=====
```

```
def do_exploration(user_id):
    u = get_user(user_id)
    if u["level"] <
EXPLORATION_MIN_LEVEL:
        return False, f"🌌
اکتشاف فقط برای سطح
{EXPLORATION_MIN_LEVEL} به بالا
.باز است."
        if u["energy"] <
EXPLORATION_ENERGY_COST:
            return False, "⚡
.انرژی کافی نداری"
            add_currency(user_id,
"energy", -
EXPLORATION_ENERGY_COST)
            lines = []
            for field, low, high in
EXPLORATION_REWARDS:
                amount =
```



```
random.randint(low, high)
    if amount > 0:

add_currency(user_id, field,
amount)
        lines.append(f"+
{amount} {field}")
        add_xp(user_id, 15)
        return True, "اکتشاف 🌌"
موفق!\n" + "\n".join(lines)
```

```
#
=====
=====
=====
# بازار سیاه
#
=====
=====
=====
```

```
def
get_black_market_today():
    today =
datetime.now().strftime("%Y-
%m-%d")
    conn = db()
    c = conn.cursor()
    c.execute("SELECT * FROM
black_market_stock WHERE
day=?", (today,))
    row = c.fetchone()
    if row:
        conn.close()
        return
BLACK_MARKET_POOL[row["item_
index"] %
len(BLACK_MARKET_POOL)]
    index =
random.randint(0,
len(BLACK_MARKET_POOL) - 1)
    c.execute("DELETE FROM
black_market_stock")
```

```
        c.execute("INSERT INTO
black_market_stock
(item_index, day) VALUES (?,
?)", (index, today))
        conn.commit()
        conn.close()
        return
BLACK_MARKET_POOL[index]
```

```
def
buy_black_market_item(user_id):
    item =
get_black_market_today()
    u = get_user(user_id)
    for currency, cost in
item["cost"].items():
        if u[currency] <
cost:
            return False,
f"❌ {currency} این کافی برای"
```

```
پیشنهاد نداری."  
    for currency, cost in  
item["cost"].items():  
  
add_currency(user_id,  
currency, -cost)  
    field, amount =  
item["give"]  
    add_currency(user_id,  
field, amount)  
    return True, f"🕵️ خرید  
موفق: {item['title']} (+  
{amount} {field})"
```

```
#  
=====
```

```
=====
```

```
=====
```

```
# فصل بازی
```

```
#
```

```
=====
```

```

=====
====

def get_season_info():
    conn = db()
    c = conn.cursor()
    c.execute("SELECT * FROM
season WHERE id=1")
    row = c.fetchone()
    conn.close()
    started =
datetime.strptime(row["start
ed_at"], "%Y-%m-%d")
    days_passed =
(datetime.now() -
started).days
    days_left = max(0,
SEASON_LENGTH_DAYS -
days_passed)
    return
row["season_number"],
days_left

```

```
def maybe_reset_season():
    number, days_left =
get_season_info()
    if days_left <= 0:
        conn = db()
        c = conn.cursor()
        c.execute(
            "UPDATE season
SET season_number=?,
started_at=? WHERE id=1",
            (number + 1,
datetime.now().strftime("%Y-
%m-%d")),
        )
        conn.commit()
        conn.close()
        return True
    return False
```

```

#
=====
=====
=====
# معامله مستقیم بین بازیکنان
#
=====
=====
=====

def
create_trade_offer(seller_id
, item_field, item_amount,
price_coin):
    u = get_user(seller_id)
    if u[item_field] <
item_amount:
        return False, "❌
موجودی کافی برای این پیشنهاد نداری."
        add_currency(seller_id,
item_field, -item_amount)
        conn = db()

```

```
c = conn.cursor()
c.execute(
    "INSERT INTO trades
(seller_id, item_field,
item_amount, price_coin,
created_at) VALUES (?, ?, ?,
?, ?)",
    (seller_id,
item_field, item_amount,
price_coin,
datetime.now().strftime("%Y-
%m-%d %H:%M:%S")),
)
conn.commit()
conn.close()
return True, "🟢 پیشنهاد
پیشنهاد
معامله ثبت شد"
```

```
def
list_open_trades(limit=10):
    conn = db()
```



```
        c = conn.cursor()
        c.execute("SELECT * FROM
trades WHERE status='open'
ORDER BY trade_id DESC LIMIT
?", (limit,))
        rows = c.fetchall()
        conn.close()
        return rows
```

```
def
accept_trade_offer(buyer_id,
trade_id):
    conn = db()
    c = conn.cursor()
    c.execute("SELECT * FROM
trades WHERE trade_id=? AND
status='open'", (trade_id,))
    trade = c.fetchone()
    if not trade:
        conn.close()
        return False, "❌ این
```

```
    "معامله دیگر در دسترس نیست"
    buyer =
get_user(buyer_id)
    if buyer["coin"] <
trade["price_coin"]:
        conn.close()
        return False, "❌"
    "کافی برای خرید نداری Coin"
    fee =
trade["price_coin"] *
TRADE_FEE_PERCENT / 100
    seller_gets =
trade["price_coin"] - fee
    add_currency(buyer_id,
"coin", -
trade["price_coin"])
    add_currency(buyer_id,
trade["item_field"],
trade["item_amount"])

add_currency(trade["seller_id"], "coin", seller_gets)
```

```
c.execute("UPDATE trades
SET status='closed',
buyer_id=? WHERE
trade_id=?", (buyer_id,
trade_id))
conn.commit()
conn.close()
return True, f"✅ معامله
!انجام شد
{trade['item_amount']}
{trade['item_field']} دریافت
کردی."
```

```
#
=====
=====
=====
# بازار پیش‌بینی قیمت
#
=====
=====
```

```
====
```

```
def
place_prediction(user_id,
direction):
    u = get_user(user_id)
    if u["coin"] <
PREDICTION_BET_AMOUNT:
        return False, "❌
Coin کافی برای شرط‌بندی نداره."
        add_currency(user_id,
"coin", -
PREDICTION_BET_AMOUNT)
        price =
get_market_price()
        conn = db()
        c = conn.cursor()
        c.execute(
            "INSERT INTO
predictions (user_id,
direction, start_price,
bet_amount, created_at)
```

```
VALUES (?, ?, ?, ?, ?)",
        (user_id, direction,
price,
PREDICTION_BET_AMOUNT,
datetime.now().strftime("%Y-
%m-%d %H:%M:%S")),
    )
    conn.commit()
    conn.close()
    return True, f" شرط ثبت  روی {direction} شد: پیش‌بینی
    قیمت {price} Coin"
```

```
def resolve_predictions():
    """ساعتی بعد از job این تابع در"""
    تغییر قیمت صدا زده می‌شود تا شرط‌های
    """باز را ببندد.
    new_price =
get_market_price()
    conn = db()
    c = conn.cursor()
```

```
        c.execute("SELECT * FROM
predictions WHERE
status='open'")
        open_bets = c.fetchall()
        results = []
        for bet in open_bets:
            won =
(bet["direction"] == "up"
and new_price >
bet["start_price"]) or (
            bet["direction"]
== "down" and new_price <
bet["start_price"]
            )
            if won:
                payout =
bet["bet_amount"] *
PREDICTION_WIN_MULTIPLIER

add_currency(bet["user_id"],
"coin", payout)
```

```

results.append((bet["user_id"], True, payout))
    else:

results.append((bet["user_id"], False, 0))
    c.execute("UPDATE
predictions SET
status='closed' WHERE
pred_id=?",
(bet["pred_id"],))
    conn.commit()
    conn.close()
    return results

```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
# فیلتر فحش
```

```
#
```

```
=====
=====
=====

def
contains_banned_word(text):
    if not text:
        return False
    lowered = text.lower()
    return any(word in
lowered for word in
BANNED_WORDS)

#
=====
=====
=====

# رقابت آنلاین (فوتبال / بسکتبال)
#
=====
=====
```



```
====
```

```
def  
get_stat_upgrade_cost(level)  
:  
    return  
int(STAT_UPGRADE_BASE_COST *  
(STAT_UPGRADE_GROWTH **  
level))
```

```
def upgrade_stat(user_id,  
sport, stat_key):  
    field = f"  
{sport}_{stat_key}"  
    u = get_user(user_id)  
    level = u[field]  
    if level >=  
STAT_MAX_LEVEL:  
        return False, "🔒 این  
مهارت به حداکثر سطح رسیده"  
    cost =
```

```
get_stat_upgrade_cost(level)
    if u["coin"] < cost:
        return False, f"❌
نیاز {cost} Coin ارتقا به
داری."
        add_currency(user_id,
"coin", -cost)
        set_field(user_id,
field, level + 1)
        return True, f"✅ مهارت
{level+1}: ارتقا یافت! سطح جدید
(هزینه: {cost} Coin)"
```

```
def get_total_power(user_id,
sport):
    u = get_user(user_id)
    stats = SPORTS[sport]
    ["stats"].keys()
    return sum(u[f"
{sport}_{stat}"] for stat in
stats)
```

```
def
get_league_tier_name(points)
:
    name = LEAGUE_TIERS[0]
[1]
    for threshold, tier_name
in LEAGUE_TIERS:
        if points >=
threshold:
            name = tier_name
    return name
```

```
#
=====
=====
=====
# اشتراک ویژه با تلگرام استارز
#
=====
```

```

=====
=====

def
activate_star_subscription(u
ser_id, tier_key, days):
    """اشتراک را برای کاربر فعال
می‌کند و تاریخ شروع/پایان را
برمی‌گرداند."""
    u = get_user(user_id)
    now = datetime.now()
    # اگر اشتراک فعلی هنوز منقضی
نشده، مدت جدید را به آن اضافه می‌کند
    current_expiry = None
    if u["vip_expires_at"]:
        try:
            current_expiry =
datetime.strptime(u["vip_exp
ires_at"], "%Y-%m-%d
%H:%M:%S")
        except ValueError:
            current_expiry =

```

None

```
        start_from =
current_expiry if
(current_expiry and
current_expiry > now) else
now
        new_expiry = start_from
+ timedelta(days=days)

        set_field(user_id,
"vip", tier_key)
        set_field(user_id,
"vip_expires_at",
new_expiry.strftime("%Y-%m-
%d %H:%M:%S"))
        return now, new_expiry

def
check_and_expire_subscriptio
n(user_id):
```

```
        """ اگر اشتراک منقضی شده باشد، """
        کاربر را به حالت بدون اشتراک
        برمی گرداند. """
        u = get_user(user_id)
        if u["vip"] != "none"
        and u["vip_expires_at"]:
            try:
                expiry =
datetime.strptime(u["vip_exp
ires_at"], "%Y-%m-%d
%H:%M:%S")
            except ValueError:
                return
            if datetime.now() >
expiry:

set_field(user_id, "vip",
"none")

set_field(user_id,
"vip_expires_at", "")
```

```

def
expire_all_subscriptions_job
():
    conn = db()
    c = conn.cursor()
    c.execute("SELECT
user_id FROM users WHERE vip
!= 'none' AND vip_expires_at
!= '')")
    rows = c.fetchall()
    conn.close()
    for row in rows:

check_and_expire_subscription(row["user_id"])

def
grant_liber_pack(user_id,
pack_key):
    pack =

```

```
STAR_LIBER_PACKS.get(pack_key)
```

```
    if not pack:
```

```
        return 0
```

```
    add_currency(user_id,  
"liber", pack["liber"])
```

```
    return pack["liber"]
```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
#   کد هدیه (Gift Code)
```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
def create_gift_code(code,  
reward_field, reward_amount,  
max_uses, created_by):
```



```
conn = db()
c = conn.cursor()
try:
    c.execute(
        "INSERT INTO
gift_codes (code,
reward_field, reward_amount,
max_uses, created_by,
created_at) VALUES (?, ?, ?,
?, ?, ?)",
        (code.upper(),
reward_field, reward_amount,
max_uses, created_by,
datetime.now().strftime("%Y-
%m-%d %H:%M:%S")),
    )
    conn.commit()
    success = True
except
sqlite3.IntegrityError:
    success = False
conn.close()
```

```
        return success

def
redeem_gift_code(user_id,
code):
    code =
code.strip().upper()
    conn = db()
    c = conn.cursor()
    c.execute("SELECT * FROM
gift_codes WHERE code=?",
(code,))
    gift = c.fetchone()
    if not gift:
        conn.close()
        return False, "❌ این
کد هدیه معتبر نیست"

    if gift["used_count"] >=
gift["max_uses"]:
        conn.close()
```

```
return False, "❌
```

ظرفیت استفاده از این کد تمام شده
است."

```
c.execute("SELECT 1 FROM  
gift_code_redemptions WHERE  
code=? AND user_id=?",  
(code, user_id))
```

```
if c.fetchone():
```

```
conn.close()
```

```
return False, "❌ قبلاً
```

این کد را استفاده کرده‌ای"

```
c.execute(  
    "INSERT INTO
```

```
gift_code_redemptions (code,
```

```
user_id, redeemed_at) VALUES  
(?, ?, ?)",  
(code, user_id,
```

```
datetime.now().strftime("%Y-  
%m-%d %H:%M:%S")),  
)
```

```
c.execute("UPDATE
gift_codes SET used_count =
used_count + 1 WHERE
code=?", (code,))
conn.commit()
conn.close()
```

```
add_currency(user_id,
gift["reward_field"],
gift["reward_amount"])
return True, f"🎁 کد هدیه +
!فعال شد
{gift['reward_amount']}
{gift['reward_field']} گرفت."
```

```
#
=====
=====
=====
# مشاور هوشمند 🤖
#
```

```

=====
=====
=====

def
get_smart_advice(user_id):
    """
    بر اساس وضعیت واقعی کاربر و
    اقتصاد، چند پیشنهاد شخصی سازی شده تولید
    می کند. """
    u = get_user(user_id)
    tips = []

    price =
get_market_price()
    if price <
MARKET_BASE_PRICE * 0.9:
        tips.append("📉 قیمت
LIBER – پایین تر از حد معمول است
")
        elif price >
MARKET_BASE_PRICE * 1.2:
            tips.append("📈 قیمت

```

اضافه LIBER بالاست - اگر LIBER
(".داری، شاید بفروشی سود کنی

```
if u["bank_deposit"] ==  
0 and u["coin"] > 500:  
    tips.append(f"  
{int(u['coin'])} Coin بدون  
استفاده داری؛ در بانک سپرده بذار تا سود  
{BANK_INTEREST_PERCENT}%  
بگیری.")
```

```
if not  
u["country_name"]:  
    tips.append(" هنوز  
کشوری نساختی! ساختنش رایگانه و بهت  
جمعیت و بودجه اولیه می‌ده  
elif u["country_pop"] >  
0:  
    tips.append(" یادت  
نره هر روز مالیات کشورت رو جمع کنی،  
می‌گیری Coin و XP رایگان")
```

```
research_info =  
get_research_info(user_id)  
if research_info and  
u["coin"] >=  
research_info["cost_coin"]:  
tips.append(f"📖  
می‌تونم همین الان تحقیق  
رو «{research_info['name']}»  
با  
{research_info['cost_coin']}  
Coin کامل کنی.")
```

```
if u["defense_level"] <  
3:  
tips.append("🛡️ سطح  
دفاعت پایینه؛ ارتقاش بده تا کشورت امن‌تر  
بشه.")
```

```
football_power =  
get_total_power(user_id,  
"football")  
basketball_power =
```

```
get_total_power(user_id,
"basketball")
    if max(football_power,
basketball_power) < 80:
        tips.append("✂ قدرت
رقابتی‌ات هنوز کمه - چند تا مهارت
ورزشی رو ارتقا بده تا تو مسابقه‌ها بیشتر
ببری.")

        today =
datetime.now().strftime("%Y-
%m-%d")
        if
u["last_daily_reward"] !=
today:
            tips.append("🎁 جایزه
روزانه‌ات رو هنوز نگرفتی - رایگانه، از
دستش نده")
            if
u["last_daily_mission"] !=
today:
                tips.append("🎯
```


مأموریت روزانه هم هنوز باز نشده، برو
") .کاملش کن

```
if u["vip"] == "none":  
    tips.append("★ با فعال  
    بیشتری XP درآمد و VIP، کردن اشتراک  
    ").می‌گیری")
```

```
if not tips:  
    tips.append("👍  
    وضعیت خیلی خوبه! همینطور با مسابقه،  
    ").صندوق و مأموریت روزانه پیش برو
```

```
return tips
```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
# TON برداشت
```

```
#
```

```

=====
=====
=====

def
create_withdraw_request(user
_id, amount, ton_address):
    u = get_user(user_id)
    if amount <
MIN_WITHDRAW_LIBER:
        return False, f"❌
حداقل مبلغ برداشت
{MIN_WITHDRAW_LIBER} LIBER
است."
    if u["liber"] < amount:
        return False, "❌
.کافی نیست LIBER موجودی

    fee = round(amount *
WITHDRAW_FEE_PERCENT / 100,
2)
    add_currency(user_id,

```

```
"liber", -amount) # مبلغ فوراً  
از حساب کسر می‌شود تا امانتی نزد  
سیستم بماند  
    conn = db()  
    c = conn.cursor()  
    c.execute(  
        "INSERT INTO  
withdrawal_requests  
(user_id, amount_liber,  
fee_liber, ton_address,  
created_at) VALUES (?, ?, ?,  
?, ?)",  
        (user_id, amount,  
fee, ton_address,  
datetime.now().strftime("%Y-  
%m-%d %H:%M:%S")),  
    )  
    request_id = c.lastrowid  
    conn.commit()  
    conn.close()  
    return True, request_id
```

```
def
list_user_withdrawals(user_i
d, limit=10):
    conn = db()
    c = conn.cursor()
    c.execute(
        "SELECT * FROM
withdrawal_requests WHERE
user_id=? ORDER BY
request_id DESC LIMIT ?",
        (user_id, limit),
    )
    rows = c.fetchall()
    conn.close()
    return rows
```

```
def
list_pending_withdrawals(lim
it=20):
    conn = db()
```

```
        c = conn.cursor()
        c.execute("SELECT * FROM
withdrawal_requests WHERE
status='pending' ORDER BY
request_id ASC LIMIT ?",
(limit,))
        rows = c.fetchall()
        conn.close()
        return rows
```

```
def
approve_withdraw_request(req
uest_id):
    conn = db()
    c = conn.cursor()
    c.execute("SELECT * FROM
withdrawal_requests WHERE
request_id=? AND
status='pending'",
(request_id,))
    req = c.fetchone()
```

```
        if not req:
            conn.close()
            return False, None
        c.execute(
            "UPDATE
withdrawal_requests SET
status='approved',
processed_at=? WHERE
request_id=?",

(datetime.now().strftime("%Y
-%m-%d %H:%M:%S"),
request_id),
        )
        conn.commit()
        conn.close()
        return True, req

def
reject_withdraw_request(requ
est_id):
```

```
conn = db()
c = conn.cursor()
c.execute("SELECT * FROM
withdrawal_requests WHERE
request_id=? AND
status='pending'",
(request_id,))
req = c.fetchone()
if not req:
    conn.close()
    return False, None
c.execute(
    "UPDATE
withdrawal_requests SET
status='rejected',
processed_at=? WHERE
request_id=?",

(datetime.now().strftime("%Y
-%m-%d %H:%M:%S"),
request_id),
)
```

```
conn.commit()
conn.close()
# بازگرداندن مبلغ به کاربر چون
درخواست رد شده

add_currency(req["user_id"],
"liber",
req["amount_liber"])
return True, req


#
=====
=====
=====

# اخبار جهانی 🌐
#
=====
=====
=====

def get_world_news():
```



```

        """ یک فید خبری زنده از """
        """ رویدادهای واقعی درون بازی می‌سازد """
        lines = []

        price =
get_market_price()
        lines.append(f"📈 قیمت
LIBER: {price} Coin" لحظه‌ای

        number, season_days_left
= get_season_info()
        lines.append(f"📅 فصل
{number} - فعلی
{season_days_left} روز تا
پایان.")

        tournament_days_left =
get_tournament_info()
        lines.append(f"🏆
{tournament_days_left} روز تا
پایان تورنمنت رقابتی فصلی
")

```

```

        item =
get_black_market_today()
        lines.append(f"🕵️ پیشنهاد
:ویژه بازار سیاه امروز
{item['title']}")

        conn = db()
        c = conn.cursor()
        c.execute("SELECT
first_name, liber FROM users
ORDER BY liber DESC LIMIT
1")
        richest = c.fetchone()
        if richest:
            lines.append(f"👑
:ثروتمندترین بازیکن این لحظه
{richest['first_name']} با
{richest['liber']} LIBER")

        c.execute("SELECT
first_name, rank_points FROM
users ORDER BY rank_points

```

```
DESC LIMIT 1")
    top_fighter =
c.fetchone()
    if top_fighter and
top_fighter["rank_points"] >
0:
        lines.append(f"✂
برترین رقابت‌گر این لحظه:
{top_fighter['first_name']} با
{top_fighter['rank_points']}
("امتیاز رنک

c.execute("SELECT
COUNT(*) as cnt FROM
matches")
    total_matches =
c.fetchone()["cnt"]
    lines.append(f"🎮 مجموع
مسابقات برگزارشده تا الان:
{total_matches}")

c.execute("SELECT
```

```
COUNT(*) as cnt FROM users  
WHERE country_name != '')
```

```
    total_countries =  
c.fetchone()["cnt"]  
    lines.append(f"🌍 تعداد  
LIBER: کشورهای ساخته شده در جهان  
{total_countries}")
```

```
    conn.close()  
    return lines
```

```
def  
join_match_queue(user_id,  
sport):  
    """اگر حریفی در صف باشد  
    بلافاصله مسابقه شبیه سازی می شود، وگرنه  
    کاربر در صف انتظار قرار می گیرد."""  
    conn = db()  
    c = conn.cursor()  
    c.execute("SELECT * FROM  
match_queue WHERE sport=?
```

```

AND user_id != ? LIMIT 1",
(sport, user_id))
    opponent_row =
c.fetchone()

    if opponent_row:
        c.execute("DELETE
FROM match_queue WHERE
user_id=?",
(opponent_row["user_id"],))
        conn.commit()
        conn.close()
        opponent_id =
opponent_row["user_id"]
        return "matched",
opponent_id
    else:
        c.execute("DELETE
FROM match_queue WHERE
user_id=?", (user_id,))
        c.execute(
            "INSERT INTO

```

```

match_queue (user_id, sport,
joined_at) VALUES (?, ?,
?),
                (user_id, sport,
datetime.now().strftime("%Y-
%m-%d %H:%M:%S")),
                )
conn.commit()
conn.close()
return "waiting",
None

```

```

def
leave_match_queue(user_id):
    conn = db()
    c = conn.cursor()
    c.execute("DELETE FROM
match_queue WHERE
user_id=?", (user_id,))
    conn.commit()
    conn.close()

```

```
def
_run_possession_battle(player
power, opponent_power,
opponent_name):
    """هسته مشترک شبیه سازی
    حمله به حمله؛ هم برای مسابقه رنک هم
    مسابقه سخت استفاده می شود"""
    player_score = 0
    opponent_score = 0
    log_lines = []
    attacker = "player"

    for possession in
range(1, MATCH_POSSESSIONS +
1):
        if attacker ==
"player":
            atk_power =
player_power +
random.randint(-10, 10)
```

```
def_power =  
opponent_power +  
random.randint(-10, 10)  
    atk_label = "شما"  
else:  
    atk_power =  
opponent_power +  
random.randint(-10, 10)  
    def_power =  
player_power +  
random.randint(-10, 10)  
    atk_label =  
opponent_name  
  
log_lines.append(f"🔵 حمله  
{possession}: {atk_label} توپ  
را ارسال کرد")  
    if atk_power >  
def_power:  
  
log_lines.append(f"⚽ گل شد !")
```



```
({atk_power} در برابر  
{def_power}))")  
        if attacker ==  
"player":  
            player_score  
+= 1  
        else:  
  
opponent_score += 1  
        elif atk_power ==  
def_power:  
  
log_lines.append(f"⚔️ قدرت  
{atk_power} مساوی بود  
دفاع در آخرین - ({def_power})  
لحظه گل را گرفت")  
        else:  
  
log_lines.append(f"🛡️ دفاع  
در برابر ({def_power}) موفق بود  
{atk_power}))")
```

```

        attacker =
"opponent" if attacker ==
"player" else "player"

        if player_score ==
opponent_score:
            log_lines.append("⌚
نتیجه مساوی شد - ضربات پنالتی
تعیین کننده...")
            if player_power +
random.randint(0, 15) >=
opponent_power +
random.randint(0, 15):
                player_score +=
1
            else:
                opponent_score
+= 1

        result = "win" if
player_score >
opponent_score else "loss"

```

```

        return player_score,
        opponent_score, result,
        log_lines

def
simulate_match(player_id,
opponent_id, sport,
vs_bot=False):
    """شبیه‌سازی مسابقه رنک عادی (با)"""
    حریف واقعی یا هوش مصنوعی
    (متعادل)"""
    player_power =
get_total_power(player_id,
sport)
    if vs_bot:
        opponent_power =
max(20, player_power +
random.randint(-15, 15))
        opponent_name = "🤖"
    "حریف هوش مصنوعی"
    else:

```

```
        opponent_power =
get_total_power(opponent_id,
sport)

        opp_user =
get_user(opponent_id)
        opponent_name =
opp_user["first_name"] or
"حريف"

        player_score,
opponent_score, result,
log_lines =
_run_possession_battle(
            player_power,
opponent_power,
opponent_name
        )

        return {
            "player_score":
player_score,
            "opponent_score":
```

```
opponent_score,  
    "result": result,  
    "log": log_lines,  
    "opponent_name":  
opponent_name,  
    "player_power":  
player_power,  
    "opponent_power":  
opponent_power,  
}
```

```
def  
simulate_hard_match(player_id,  
sport):  
    """/ شبیه سازی مسابقه سخت  
    پرمخاطره برابر حریف هوش مصنوعی  
    قوی تر. """  
    player_power =  
get_total_power(player_id,  
sport)  
    opponent_power = max(30,
```

```
int(player_power *
HARD_MATCH_OPPONENT_BOOST) +
random.randint(0, 20))
    opponent_name = "🔥 حریف
سخت (هوش مصنوعی)"

    player_score,
    opponent_score, result,
    log_lines =
    _run_possession_battle(
        player_power,
        opponent_power,
        opponent_name
    )

    return {
        "player_score":
        player_score,
        "opponent_score":
        opponent_score,
        "result": result,
        "log": log_lines,
```

```
        "opponent_name":  
opponent_name,  
        "player_power":  
player_power,  
        "opponent_power":  
opponent_power,  
    }
```

```
def  
apply_match_result(player_id  
, opponent_id, sport,  
match_data, vs_bot=False):  
    conn = db()  
    c = conn.cursor()  
    c.execute(  
        "INSERT INTO matches  
(player_id, opponent_id,  
sport, player_score,  
opponent_score, result, log,  
created_at) VALUES (?, ?, ?,  
?, ?, ?, ?, ?)",
```

```
(
    player_id,
    opponent_id if
not vs_bot else 0,
    sport,

    match_data["player_score"],

    match_data["opponent_score"]
    ,

    match_data["result"],

    json.dumps(match_data["log"]
    , ensure_ascii=False),

    datetime.now().strftime("%Y-
    %m-%d %H:%M:%S"),
    ),
)
conn.commit()
conn.close()
```



```
        add_currency(player_id,
"matches_played", 1)
        if not vs_bot:

add_currency(opponent_id,
"matches_played", 1)

        pot = MATCH_ENTRY_FEE *
(1 if vs_bot else 2)
        fee = pot *
MATCH_POT_FEE_PERCENT / 100
        prize = round(pot - fee,
2)

        if match_data["result"]
== "win":

add_currency(player_id,
"liber", prize)

add_currency(player_id,
```

```
"matches_won", 1)

add_currency(player_id,
"rank_points",
RANK_WIN_POINTS)
    if not vs_bot:

add_currency(opponent_id,
"rank_points",
RANK_LOSS_POINTS)
    else:
        if not vs_bot:

add_currency(opponent_id,
"liber", prize)

add_currency(opponent_id,
"matches_won", 1)

add_currency(opponent_id,
"rank_points",
RANK_WIN_POINTS)
```

```
add_currency(player_id,  
"rank_points",  
RANK_LOSS_POINTS)
```

```
    # جلوگیری از منفی شدن امتیاز  
    رنک  
    for uid in ([player_id]  
if vs_bot else [player_id,  
opponent_id]):  
        u2 = get_user(uid)  
        if u2["rank_points"]  
< 0:  
            set_field(uid,  
"rank_points", 0)
```

```
def get_tournament_info():  
    conn = db()  
    c = conn.cursor()  
    c.execute("SELECT * FROM  
tournament WHERE id=1")
```

```
        row = c.fetchone()
        conn.close()
        started =
datetime.strptime(row["start
ed_at"], "%Y-%m-%d")
        days_passed =
(datetime.now() -
started).days
        days_left = max(0,
TOURNAMENT_LENGTH_DAYS -
days_passed)
        return days_left
```

```
async def
maybe_resolve_tournament(bot
):
    days_left =
get_tournament_info()
    if days_left > 0:
        return
    await
```

```
_resolve_tournament_now(bot)
```

```
async def  
maybe_resolve_tournament_for  
ced(bot):  
    await  
_resolve_tournament_now(bot)
```

```
async def  
_resolve_tournament_now(bot)  
:  
    conn = db()  
    c = conn.cursor()  
    c.execute("SELECT  
user_id, first_name,  
rank_points FROM users ORDER  
BY rank_points DESC LIMIT  
5")  
    top5 = c.fetchall()  
    conn.close()
```

```
        for i, row in
enumerate(top5, start=1):
    if
row["rank_points"] <= 0:
        continue
        liber_reward =
TOURNAMENT_REWARDS.get(i, 0)
        medal_reward =
TOURNAMENT_MEDAL_REWARDS.get
(i, 0)
        if liber_reward:

add_currency(row["user_id"],
"liber", liber_reward)
        if medal_reward:

add_currency(row["user_id"],
"medal", medal_reward)
        if liber_reward or
medal_reward:
            reward_text = f"
```

```

{liber_reward} LIBER" if
liber_reward else f"
{medal_reward} مدال"
        try:
            await
bot.send_message(

row["user_id"],
                                f"🏆
را {i} تبریک! در تورنمنت فصلی رتبه
جایزه {reward_text} کسب کردی و
گرفتی!",
                                )
        except
Exception:
            pass

conn = db()
c = conn.cursor()
c.execute("UPDATE
tournament SET started_at=?
WHERE id=1",

```

```
(datetime.now().strftime("%Y-%m-%d"),))
```

```
    c.execute("UPDATE users  
SET rank_points=0")  
    conn.commit()  
    conn.close()
```

```
def  
get_competition_leaderboard(  
limit=10):  
    conn = db()  
    c = conn.cursor()  
    c.execute(  
        "SELECT first_name,  
rank_points, matches_played,  
matches_won FROM users "  
        "WHERE rank_points >  
0 ORDER BY rank_points DESC  
LIMIT ?",  
        (limit,),  
    )
```



```
        rows = c.fetchall()
        conn.close()
        return rows

def
get_recent_matches(limit=5):
    conn = db()
    c = conn.cursor()
    c.execute(
        "SELECT m.*,
u1.first_name as
player_name, u2.first_name
as opponent_name "
        "FROM matches m "
        "LEFT JOIN users u1
ON m.player_id = u1.user_id
"
        "LEFT JOIN users u2
ON m.opponent_id =
u2.user_id "
        "ORDER BY m.match_id
```

```
DESC LIMIT ?",
        (limit,),
    )
    rows = c.fetchall()
    conn.close()
    return rows
```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
# کیبوردھا
```

```
#
```

```
=====
```

```
=====
```

```
=====
```

```
def
main_menu_keyboard(user_id=N
one):
    today =
```

```

datetime.now().strftime("%Y-
%m-%d")
    daily_badge = ""
    mission_badge = ""
    u = get_user(user_id) if
user_id else None
    if u:
        if
u["last_daily_reward"] !=
today:
            daily_badge = "
🔴"
        if
u["last_daily_mission"] !=
today:
            mission_badge =
" 🔴"

    buttons = [

[InlineKeyboardButton("👤
پروفایل من",

```

```
callback_data="profile"),
```

```
InlineKeyboardButton("🌐  
امپراتوری من",  
callback_data="country"),
```

```
InlineKeyboardButton("📈 بازار  
زنده",  
callback_data="market")],
```

```
[InlineKeyboardButton("💰  
گنجینه من",  
callback_data="wallet"),
```

```
InlineKeyboardButton("🏦 بانک  
مرکزی", callback_data="bank"),
```

```
InlineKeyboardButton("🏠  
فروشگاه ویژه",  
callback_data="shop")],
```

```
[InlineKeyboardButton("🎁
```

```
"صندوق شانس",
callback_data="chests"),

InlineKeyboardButton(f"🎯
مأموریت روزانه {mission_badge}",
callback_data="missions"),

InlineKeyboardButton("🏆 لیگ
من",
callback_data="league")],

[InlineKeyboardButton("🤝
اتحاد قدرت",
callback_data="alliance"),

InlineKeyboardButton("📊
برترین‌ها",
callback_data="ranking"),

InlineKeyboardButton("★
VIP عضویت",
callback_data="vip")],
```

```
[InlineKeyboardButton("👥  
زیرمجموعه گیری",  
callback_data="invite"),
```

```
InlineKeyboardButton(f"🎁  
جایزه ورود روزانه {daily_badge}",  
callback_data="daily"),
```

```
InlineKeyboardButton("📰 اخبار  
جهان",  
callback_data="news")],
```

```
[InlineKeyboardButton("🏆  
دستاوردهای من",  
callback_data="achievements"  
),
```

```
InlineKeyboardButton("🔬  
آزمایشگاه فناوری",  
callback_data="research"),
```

```
InlineKeyboardButton("🛡️ قدرت  
دفاعی",  
callback_data="defense")],
```

```
[InlineKeyboardButton("🌌  
اکتشاف سرزمین",  
callback_data="exploration")  
,
```

```
InlineKeyboardButton("🕵️ بازار  
سیاه امروز",  
callback_data="black_market"  
),
```

```
InlineKeyboardButton("📅 فصل  
بازی",  
callback_data="season")],
```

```
[InlineKeyboardButton("📦 بازار  
بازیکنان",  
callback_data="p2p_market"),
```

```
InlineKeyboardButton("📄 حدس  
"قیمت",  
callback_data="prediction")]  
,
```

```
[InlineKeyboardButton("🔪  
"آوردگاه رقابت",  
callback_data="competition")  
],
```

```
[InlineKeyboardButton("🤖  
"مشاور هوشمند",  
callback_data="smart_advisor"  
"),
```

```
InlineKeyboardButton("📁 کد  
"هدیه",  
callback_data="gift_code")],
```

```
[InlineKeyboardButton("❓  
"راهنمای کامل",  
callback_data="help"),
```



```

InlineKeyboardButton("📞
پشتیبانی",
callback_data="support")],
]
    if user_id in ADMIN_IDS:
        pending_count =
len(list_pending_withdrawals
())
        admin_badge = f"
🔴{pending_count}" if
pending_count else ""

buttons.append([InlineKeyboa
rdButton(f"👑 پنل مدیریت
TITAN{admin_badge}",
callback_data="admin_panel")
])
    return
InlineKeyboardMarkup(buttons
)

```

```
def
back_keyboard(target="back_m
ain"):
    return
InlineKeyboardMarkup([[Inlin
eKeyboardButton("بازگشت به ←
BACK",
callback_data=target)]])
```

```
#
=====
=====
=====
# عضویت اجباری
#
=====
=====
=====
```

```
async def
```

```
check_force_join(update:
Update, context:
ContextTypes.DEFAULT_TYPE,
user_id) -> bool:
    if not
FORCE_JOIN_CHANNELS:
        return True
    not_joined = []
    for ch in
FORCE_JOIN_CHANNELS:
        try:
            member = await
context.bot.get_chat_member(
ch["id"], user_id)
            if member.status
in ("left", "kicked"):

not_joined.append(ch)
        except Exception as
e:

logger.warning(f"Force join
```

```
check failed for {ch['id']}:  
{e}")
```

```
not_joined.append(ch)
```

```
if not_joined:  
    buttons = [
```

```
[InlineKeyboardButton(f"  
{ch['title']}"  
    url=ch["url"])]
```

```
        for ch in  
not_joined  
    ]
```

```
buttons.append([InlineKeyboa  
rdButton(" عضو شدم، بررسی  
مجدد",  
callback_data="check_join")]  
)
```

```
text = " برای استفاده  
ابتدا باید در کانالهای LIBER از ربات
```

```
زیر عضو شوی:"
    if update.message:
        await
update.message.reply_text(te
xt,
reply_markup=InlineKeyboardM
arkup(buttons))
    elif
update.callback_query:
        await
update.callback_query.messag
e.reply_text(text,
reply_markup=InlineKeyboardM
arkup(buttons))
        return False
    return True
```

```
#
=====
=====
=====
```

```

# /start
#
=====
=====
=====

async def start(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    tg_user =
update.effective_user
    user_id = tg_user.id

    ref_by = 0
    if context.args:
        try:
            possible_ref =
int(context.args[0])
            if possible_ref
!= user_id:
                ref_by =
possible_ref

```

```
except ValueError:
    pass

    if is_banned(user_id):
        await
        update.message.reply_text("
        حساب شما مسدود شده است. برای
        اطلاعات بیشتر با پشتیبانی تماس
        بگیرید.")
        return

        joined_ok = await
        check_force_join(update,
        context, user_id)
        if not joined_ok:
            return

            is_new =
            create_user_if_not_exists(tg
            _user, ref_by)
            u = get_user(user_id)
```

```

now = datetime.now()
current_time =
now.strftime("%H:%M:%S")
current_date =
now.strftime("%Y-%m-%d")
last_year = now.year - 1

welcome = (
    "🌌
===== 🌌\n"
    f"🌟 خوش اومدی به دنیای
<b>LIBER</b> 🌟\n"
    "🌌
===== 🌌
\n\n"
    f"<b>👋 سلام جناب {tg_user.first_name}</b> خوش
خوش اومدی به لیبر
    f"<code>👤 آیدی عددی: {user_id}</code>\n"
    f"👤 نام کاربری:
@{tg_user.username if

```



```

tg_user.username else
'ندارد'}\n"
    f"📅 ورود:
{current_date}\n"
    f"🕒 ساعت فعلی:
{current_time}\n"
    f"📅 یک سال قبل:
{last_year}\n"
    f"🔄 تعداد ورود:
{u['login_count']}\n\n"
    f"💰 موجودی فعلی:\n"
    f"👆 LIBER: <b>
{u['liber']}</b>\n"
    f"💵 Coin: <b>
{u['coin']}</b>\n"
    f"⚡ Energy: <b>
{u['energy']}</b>\n"
    f"💎 Diamond: <b>
{u['diamond']}</b>\n"
    f"🏅 Medal: <b>
{u['medal']}</b>\n\n"
    "⚠️ توجه: تقلب، فحاشی و

```

اسپم در ربات ممنوع است و باعث اختار
.\n\n" یا مسدودی می‌شود

از دکمه‌های زیر برای 📌"
:"شروع بازی استفاده کن
)

```
if is_new:
```

```
welcome += "\n\n🎉"
```

پیوستی، ۱۰۰ LIBER چون تازه به
!"هدیه گرفتی Coin و ۵۰۰ LIBER

```
await
```

```
update.message.reply_text(welcome,  
parse_mode=ParseMode.HTML,  
reply_markup=main_menu_keyboard(user_id))
```

```
#
```

```
=====
```

```
=====
```

```

=====
#   ساعتی نوسان بازار Job
#
=====
=====

=====

async def
hourly_market_job(context:
ContextTypes.DEFAULT_TYPE):
    price =
get_market_price()
    change =
random.uniform(*MARKET_FLUCT
UATION_RANGE)
    new_price = max(1,
round(price * (1 + change),
2))

set_market_price(new_price)
    direction = "صعودی" if
new_price >= price else "

```

نزولی"

```
logger.info(f"Market  
price updated: {price} ->  
{new_price} ({direction})")
```

```
results =  
resolve_predictions()  
for target_user_id, won,  
payout in results:  
    try:  
        if won:  
            text = f"🎉  
!پیش‌بینی‌ات درست بود  
{round(payout,2)} Coin گرفتی."  
        else:  
            text = "😞  
."پیش‌بینی‌ات این‌بار درست نبود  
        await  
context.bot.send_message(target_user_id, text)  
    except Exception as  
e:
```

```
logger.warning(f"Could not  
notify user  
{target_user_id}: {e}")
```

```
        maybe_reset_season()  
        await  
maybe_resolve_tournament(con  
text.bot)
```

```
expire_all_subscriptions_job  
( )
```

```
#  
=====  
=====  
=====  
# هندلر اصلی دکمه ها  
#  
=====  
=====
```

```
====
```

```
async def
button_handler(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    query =
update.callback_query
    user_id =
query.from_user.id

    if is_banned(user_id):
        await
query.answer("حساب شما  مسدود است", show_alert=True)
        return

    ok, warns =
anti_spam_check(user_id)
    if not ok:
        if warns >= 5:
            conn = db()
```

```

        c =
conn.cursor()

c.execute("UPDATE users SET
warn_count=warn_count+1
WHERE user_id=?",
(user_id,))

        conn.commit()
        conn.close()

        await
query.answer("⚠️ آرام‌تر! لطفاً کمی آرام‌تر! (ضد اسپم فعال شد)",
show_alert=True)
        return

        # تشخیص هوشمند الگوی کلیک
ربات‌گونه (فراتر از حد طبیعی یک انسان)
        if
check_click_rate_cheat(user_
id):

            await
flag_suspicious_activity(use

```

```
r_id, context, "الگوی کلیک"
("غیرطبیعی/اسکرپتی
    if
is_banned(user_id):
    await
query.answer("حساب شما به ❌
    دلایل فعالیت مشکوک مسدود شد",
show_alert=True)
    return
    await
query.answer("فعالیت غیرطبیعی ⚠️
    تشخیص داده شد. کمی آرام تر پیش
    برو.", show_alert=True)
    return

    await query.answer()
    data = query.data

    if data == "check_join":
        joined_ok = await
check_force_join(update,
context, user_id)
```



```
        if joined_ok:
            await
            query.message.edit_text("✅  
عضویت شما تایید شد! از منوی زیر  
استفاده کن",  
reply_markup=main_menu_keyboard(user_id))
            return

        if data == "back_main":
            await
            query.message.edit_text("🌐  
LIBER: منوی اصلی",  
reply_markup=main_menu_keyboard(user_id))
            return

        u = get_user(user_id)

        # ----- پروفایل
        -----

        if data == "profile":
```

```
league =
get_league_name(u["xp"] +
u["level"] * XP_PER_LEVEL)
comp_league =
get_league_tier_name(u["rank
_points"])
username_display =
f"@{u['username']}" if
u["username"] else "ثبت نشده"
bio_display =
u["bio"] or "بیوگرافی ثبت نشده"
(setbio (متن بنویس))
text = (
    "👤 <b>پروفایل</b>\n
شما</b>\n"

    "_____ \n"
    f"👤 نام کاربری:
{username_display}\n"
    f"🆔 آیدی عددی:
<code>{user_id}</code>\n"
    f"🏠 نام:
```

```
{u['first_name']}\n"
    f"📄 بيو:
{bio_display}\n"

"_____\n"
    f"★ سطح:
{u['level']}\n | 💎 XP:
{u['xp']}/{u['level']*XP_PER
_LEVEL}\n"

    f"🏆 لیگ اقتصادی:
{league}\n"

    f"⚔️ لیگ رقابتی:
{comp_league}
({u['rank_points']}\n امتیاز)\n"
    f"🏅 مدال:
{u['medal']}\n"

    f"👑 VIP: اشتراک
{u['vip'] if u['vip'] !=
'none' else 'ندارد'}\n"

"_____\n"
    f"👤 موجودی
```

```

LIBER: {u['liber']}\n"
        f"🌐 کشور:
        {u['country_name']} or 'ثبت
        نشده'\n"

        f"👥 زیرمجموعه ها:
        {u['ref_count']}\n نفر"

        f"⚠️ اخطارها:
        {u['warn_count']}/{MAX_WARN_
        BEFORE_BAN}\n"

        "\n"

        f"📅 تاریخ عضویت:
        {u['joined_at']}\n"

        f"🔄 تعداد ورود به
        ربات: {u['login_count']}"
    )
    await
    query.message.edit_text(text
    , parse_mode=ParseMode.HTML,
    reply_markup=back_keyboard()
    )

```

```

# ----- کیف
پول -----
elif data == "wallet":
    text = (
        "💰 <b>کیف پول</b>
شما</b>\n\n"
        f"👆 LIBER:
{u['liber']}\n"
        f"💵 Coin:
{u['coin']}\n"
        f"⚡ Energy:
{u['energy']}\n"
        f"💎 Diamond:
{u['diamond']}\n"
        f"🏆 Medal:
{u['medal']}"
    )
    kb =
InlineKeyboardMarkup([

[InlineKeyboardButton("📡
برداشت",

```

```

callback_data="withdraw"),

InlineKeyboardButton("📥  
واریز",
callback_data="deposit")],

[InlineKeyboardButton("⬅️  
BACK  
بازگشت به منو",
callback_data="back_main")],
    ])
    await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=kb)

# ----- بازار -----
-----

elif data == "market":
    price =
get_market_price()
    text = (
        "📈 <b>بازار

```

```

LIBER</b>\n\n"
        قیمت لحظه‌ای هر 
LIBER: <b>{price}</b>
Coin</b>\n"
        قیمت هر ۱ ساعت 
        .\n"
        موجودی شما :
        {u['liber']} LIBER |
        {u['coin']} Coin"
    )
    kb =
    InlineKeyboardMarkup([

    [InlineKeyboardButton("🟢
خرید",
callback_data="market_buy"),

    InlineKeyboardButton("🔴
فروش",
callback_data="market_sell")
],

```

```
[InlineKeyboardButton("←  
BACK",  
    "بازگشت به منو",  
    callback_data="back_main")],  
    ])  
    await  
    query.message.edit_text(text  
    , parse_mode=ParseMode.HTML,  
    reply_markup=kb)  
  
    elif data in  
    ("market_buy",  
    "market_sell"):  
        action = "buy" if  
data == "market_buy" else  
"sell"  
        amounts = [10, 50,  
100, 500]  
        buttons = [  
  
[InlineKeyboardButton(f"  
{amt} LIBER",  
    callback_data=f"
```



```

{action}_{amt}") for amt in
amounts[:2]],

[InlineKeyboardButton(f"
{amt} LIBER",
callback_data=f"
{action}_{amt}") for amt in
amounts[2:]],

[InlineKeyboardButton("←
BACK
بازگشت",
callback_data="market")],
]
label = "خرید" if
action == "buy" else "فروش"
await
query.message.edit_text(f"$🔄
مقدار {label} کن را انتخاب کن:",
reply_markup=InlineKeyboardM
arkup(buttons))

elif

```

```

data.startswith("buy_") or
data.startswith("sell_"):
    action, amt_str =
data.split("_")
    amount =
float(amt_str)
    price =
get_market_price()
    cost = amount *
price

    if action == "buy":
        fee = cost *
BUY_FEE_PERCENT / 100
        total_cost =
cost + fee
        if u["coin"] <
total_cost:
            await
query.message.edit_text("❌
کافی نیست Coin موجودی",
reply_markup=back_keyboard())

```

```

)

        return

    add_currency(user_id,
        "coin", -total_cost)

    add_currency(user_id,
        "liber", amount)

    add_currency(user_id,
        "trade_count", 1)

    unlocked =
    check_achievements(user_id)
    text = f"✅ خرید
    خریدی به {amount} LIBER! موفق
    Coin (+ {round(cost,2)} قیمت
    کارمزد {round(fee,2)})"
    if unlocked:
        text +=
        "\n\n🏆 دستاورد جدید: " + ",
    ".join(unlocked)
    await

```

```
query.message.edit_text(text
,
reply_markup=back_keyboard()
)
        else:
            if u["liber"] <
amount:
                await
query.message.edit_text("❌
کافی نیست LIBER موجودی",
reply_markup=back_keyboard()
)
                return
                fee = cost *
SELL_FEE_PERCENT / 100
                net = cost - fee

add_currency(user_id,
"liber", -amount)

add_currency(user_id,
"coin", net)
```

```

add_currency(user_id,
"trade_count", 1)
        unlocked =
check_achievements(user_id)
        text = f"✅ فروش
فروختی و {amount} LIBER! موفق
{round(net,2)} Coin گرفتی
(کارمزد {round(fee,2)})"
        if unlocked:
            text +=
"\n\n🏆 دستاورد جدید: " + ",
".join(unlocked)
            await
query.message.edit_text(text
,
reply_markup=back_keyboard()
)

# ----- بانک --
-----
elif data == "bank":

```

```

text = (
    "  <b>بانک</b>
LIBER</b>\n\n"
    f"  سپرده فعلی:
{u['bank_deposit']} Coin\n"
    f"  سود روزانه
:سپرده:
{BANK_INTEREST_PERCENT}%\n"
    f"  وام فعلی:
{u['loan_amount']} Coin\n"
    f"  کارمزد وام:
{LOAN_INTEREST_PERCENT}%
)
kb =
InlineKeyboardMarkup([

[InlineKeyboardButton("✚
سپرده گذاری",
callback_data="bank_deposit"
),

InlineKeyboardButton("✚

```

```
برداشت سپرده",  
callback_data="bank_withdraw  
")],
```

```
[InlineKeyboardButton("🏠  
درخواست وام",  
callback_data="bank_loan"),
```

```
InlineKeyboardButton("✅  
پرداخت وام",  
callback_data="bank_payloan"  
)],
```

```
[InlineKeyboardButton("⬅️  
بازگشت به منو",  
callback_data="back_main")],  
])
```

```
await  
query.message.edit_text(text  
, parse_mode=ParseMode.HTML,  
reply_markup=kb)
```

```
elif data ==
"bank_deposit":
    amount = min(500,
u["coin"])
    if amount <= 0:
        await
query.message.edit_text("❌
کافی نیست Coin موجودی",
reply_markup=back_keyboard("
bank"))

    return

add_currency(user_id,
"coin", -amount)

add_currency(user_id,
"bank_deposit", amount)
    unlocked =
check_achievements(user_id)
    text = f"✅ {amount}
Coin به سپرده بانکی اضافه شد"
    if unlocked:
```



```
        text += "\n\n🏆  
دست‌آورد جدید: " + "  
".join(unlocked)  
        await  
        query.message.edit_text(text  
,  
        reply_markup=back_keyboard("bank"))  
  
        elif data ==  
        "bank_withdraw":  
            u2 =  
            get_user(user_id)  
            amount =  
            u2["bank_deposit"]  
            if amount <= 0:  
                await  
                query.message.edit_text("❌  
سپرده‌ای برای برداشت وجود ندارد",  
                reply_markup=back_keyboard("bank"))  
  
            return
```

```
        interest = amount *
BANK_INTEREST_PERCENT / 100
        total = amount +
interest
        set_field(user_id,
"bank_deposit", 0)

add_currency(user_id,
"coin", total)
        await
query.message.edit_text(
        f"✅ کل سپرده
برداشت شد: {round(total,2)}
Coin (شامل
{round(interest,2)} سود)",

reply_markup=back_keyboard("
bank"),
        )

elif data ==
"bank_loan":
```

```
        max_loan =
u["level"] * 100 *
MAX_LOAN_MULTIPLIER
        if u["loan_amount"]
> 0:
            await
query.message.edit_text("❌
ابتدا وام فعلی را پرداخت کن
",
reply_markup=back_keyboard("
bank"))
            return
            loan = max_loan

add_currency(user_id,
"coin", loan)
            set_field(user_id,
"loan_amount", loan * (1 +
LOAN_INTEREST_PERCENT /
100))
            await
query.message.edit_text(
            f"✅ وام {loan}
```

```
دریافت شد. مبلغ بازپرداخت با Coin  
کارمزد: {round(loan*  
(1+LOAN_INTEREST_PERCENT/100  
,2)} Coin",  
  
reply_markup=back_keyboard("bank"),  
    )  
  
    elif data ==  
    "bank_payloan":  
        u2 =  
        get_user(user_id)  
        if u2["loan_amount"]  
        <= 0:  
            await  
            query.message.edit_text("✅  
شما وامی ندارید",  
            reply_markup=back_keyboard("bank"))  
            return  
            if u2["coin"] <
```

```
u2["loan_amount"]:  
    await  
    query.message.edit_text("❌  
برای پرداخت کامل وام Coin موجودی  
کافی نیست",  
reply_markup=back_keyboard("bank"))  
    return  
  
add_currency(user_id,  
"coin", -u2["loan_amount"])  
    set_field(user_id,  
"loan_amount", 0)  
    await  
    query.message.edit_text("✅  
وام با موفقیت پرداخت شد",  
reply_markup=back_keyboard("bank"))  
  
# ----- کشور -----  
-----  
    elif data == "country":
```

```
        if not
u["country_name"]:
            text = "🌐 هنوز
کشوری نساخته‌ای! با ثبت کشور، جمعیت
ش.و بودجه اولیه می‌گیری"
            kb =
InlineKeyboardMarkup([

[InlineKeyboardButton("🏛️
ساخت کشور",
callback_data="country_creat
e")],

[InlineKeyboardButton("⬅️
BACK
بازگشت به منو",
callback_data="back_main")],
            ])
            await
query.message.edit_text(text
, reply_markup=kb)
        else:
            text = (
```

```

        f"🌐 <b>کشور</b>
        {u['country_name']}</b>\n\n"
        f"👥 جمعیت:
        {u['country_pop']}\n"
        f"💰 بودجه:
        {u['country_budget']}\n"
        Coin\n"

        "📈 هر بار
        «جمع‌آوری مالیات» بزنی، بر اساس جمعیت
        درآمد می‌گیری."
    )
    kb =
    InlineKeyboardMarkup([

    [InlineKeyboardButton("💰
    جمع‌آوری مالیات",
    callback_data="country_tax")
    ],

    [InlineKeyboardButton("⬅️
    بازگشت به منو",
    callback_data="back_main")],

```

```
] )
    await
    query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=kb)

    elif data ==
"country_create":
        set_field(user_id,
"country_name", f"کشور
{u['first_name']}")
        set_field(user_id,
"country_pop", 100)
        set_field(user_id,
"country_budget", 200)
        unlocked =
check_achievements(user_id)
        text = " 🏆 ساختہ
کشورت Coin شد! ۱۰۰ جمعیت و ۲۰۰
بودجه .اوليه گرفتى"
        if unlocked:
            text += "\n\n 🏆
```



```
دستاورد جديد: " + ",
".join(unlocked)
    await
query.message.edit_text(text
,
reply_markup=back_keyboard()
)

    elif data ==
"country_tax":
        u2 =
get_user(user_id)
        tax_income =
round(u2["country_pop"] *
0.5, 2)

add_currency(user_id,
"country_budget",
tax_income)

add_currency(user_id,
"coin", tax_income)
```

```

        new_level =
add_xp(user_id, 5)
        await
query.message.edit_text(
            f"💰 مالیات جمع آوری
شد: {tax_income} Coin\n★ 5
XP (سطح فعلی) گرفتی:
{new_level})",

reply_markup=back_keyboard()
,
    )

# ----- صندوقها
-----
elif data == "chests":
    buttons = []
    row = []
    for i, key in
enumerate(CHEST_TABLE.keys())
):

```

```
row.append(InlineKeyboardBut
ton(f"📁 {key}",
callback_data=f"chest_{key}"
))

        if len(row) ==
3:

buttons.append(row)
        row = []
        if row:

buttons.append(row)

buttons.append([InlineKeyboa
rdButton("⬅️ بازگشت به منو",
callback_data="back_main")])
        await
query.message.edit_text("📁
یک صندوق را برای باز کردن انتخاب
کن:",
reply_markup=InlineKeyboardM
arkup(buttons))
```

```
elif
data.startswith("chest_"):
    key =
data.split("_", 1)[1]
    chest =
CHEST_TABLE.get(key)
    if not chest:
        await
query.message.edit_text("❌
صندوق نامعتبر.",
reply_markup=back_keyboard("
chests"))
        return
    for currency, cost
in chest["cost"].items():
        if u[currency] <
cost:
            await
query.message.edit_text(f"❌
{currency} کافی برای باز کردن این
صندوق نداری.",
```

```
reply_markup=back_keyboard("
    chests"))

        return
        for currency, cost
in chest["cost"].items():

add_currency(user_id,
currency, -cost)

        reward_lines = []
        for field, low, high
in chest["rewards"]:
            amount =
random.randint(low, high)
            if field ==
"xp":

add_xp(user_id, amount)
            else:

add_currency(user_id, field,
amount)
```

```
reward_lines.append(f"+
{amount} {field}")

add_currency(user_id,
"chest_count", 1)
unlocked =
check_achievements(user_id)
text = f"🎉 صندوق
{key} باز شد!\n\n" +
"\n".join(reward_lines)
if unlocked:
text += "\n\n🏆
دستآورد جدید: " + ",
".join(unlocked)
await
query.message.edit_text(text
,
reply_markup=back_keyboard("
chests"))
```

```

# -----
# -----
elif data == "missions":
    today =
datetime.now().strftime("%Y-
%m-%d")
    done_today =
u["last_daily_mission"] ==
today
    text = (
        "🎯 <b>مأموریت</b>\n\n"
        "📋 وظیفه: با ربات
        تعامل داشته باش (باز کردن این منو
        \nکافیست)
        f"🎁 جایزه:
        {DAILY_MISSION_XP} XP +
        {DAILY_MISSION_LIBER}
        LIBER\n\n"
        + ("✅ امروز قبلاً"
        if done_today
        else "🟡 آماده دریافت جایزه")

```

```
)
kb_buttons = []
if not done_today:

    kb_buttons.append([InlineKeyboardButton(
        "✅ دریافت جایزه",
        callback_data="mission_claim"
    )])

    kb_buttons.append([InlineKeyboardButton(
        "⬅️ بازگشت به منو",
        callback_data="back_main"
    )])
    await
    query.message.edit_text(text
        , parse_mode=ParseMode.HTML,
        reply_markup=InlineKeyboardMarkup(
            kb_buttons))

    elif data ==
    "mission_claim":
        today =
        datetime.now().strftime("%Y-
```



```
%m-%d")
        if
u["last_daily_mission"] ==
today:
            await
query.message.edit_text("✅
امروز قبلاً دریافت کردی
reply_markup=back_keyboard("
missions"))
            return
            set_field(user_id,
"last_daily_mission", today)

add_currency(user_id,
"liber",
DAILY_MISSION_LIBER)
            new_level =
add_xp(user_id,
DAILY_MISSION_XP)
            await
query.message.edit_text(
            f"🎉 جایزه مأموریت
```

```
{DAILY_MISSION_XP}  
روزانه گرفتی  
XP + {DAILY_MISSION_LIBER}  
LIBER\n(سطح فعلی:  
{new_level})",
```

```
reply_markup=back_keyboard("  
missions"),  
)
```

```
# ----- لیگ --  
-----  
elif data == "league":  
    total_xp = u["xp"] +  
u["level"] * XP_PER_LEVEL  
    league =  
get_league_name(total_xp)  
    text = (  
        "🏆 <b>لیگ  
شما</b>\n\n"  
        f"🏅 لیگ فعلی:  
{league}\n"  
        f"💎 XP: مجموع
```

```
{total_xp}\n\n"
```

بیشتر XP با کسب 

(مأموریت، بازار، مالیات) به لیگ بالاتر
می‌روی."

```
)
```

```
await
```

```
query.message.edit_text(text  
, parse_mode=ParseMode.HTML,  
reply_markup=back_keyboard()  
)
```

```
# ----- رتبه‌بندی
```

```
-----
```

```
elif data == "ranking":  
    conn = db()  
    c = conn.cursor()  
    c.execute("SELECT  
first_name, liber FROM users  
ORDER BY liber DESC LIMIT  
10")  
  
    top = c.fetchall()  
    conn.close()
```

```

        lines = [f"{i+1}.
{row['first_name']} -
{row['liber']} LIBER" for i,
row in enumerate(top)]
        text = "📊 <b>برترین‌های
LIBER</b>\n\n" +
"\n".join(lines) if lines
else ".هنوز داده‌ای موجود نیست"
        await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=back_keyboard()
)

# ----- VIP -----
-----
elif data == "vip":

check_and_expire_subscription(
user_id)
u =
get_user(user_id)

```

```
        expiry_line = ""
        if u["vip"] !=
"none" and
u["vip_expires_at"]:
            expiry_line =
f"\n\n🕒 اشتراک فعلی تو
({u['vip']}) تا
{u['vip_expires_at']} معتبر
است."

        text = (
            "★ <b>سطوح
VIP</b>\n\n"
            + "\n".join(
                f"{tier}:
{info['cost_diamond']}
Diamond – درآمد و
x{info['income_bonus']}"
                for tier,
info in VIP_TIERS.items()
            )
            + expiry_line
            + "\n\n🌟 همچنین
```

می‌تونی با تلگرام استارز اشتراک ویژه با
مزایای بیشتر بخری

)

buttons =

```
[[InlineKeyboardButton(f"خرید  
{tier}",  
callback_data=f"vip_{tier}")  
] for tier in VIP_TIERS]
```

```
buttons.append([InlineKeyboa  
rdButton("★ با  
استارز",  
callback_data="star_vip_menu  
")])
```

```
buttons.append([InlineKeyboa  
rdButton("←  
BACK بازگشت به منو",  
callback_data="back_main")])
```

await

```
query.message.edit_text(text  
, parse_mode=ParseMode.HTML,  
reply_markup=InlineKeyboardM
```

```
arkup(buttons))

    elif
data.startswith("vip_") and
not
data.startswith("vip_star"):
    tier =
data.split("_", 1)[1]
    info =
VIP_TIERS.get(tier)
    if not info:
        await
query.message.edit_text("❌
نامعتبر VIP سطح",
reply_markup=back_keyboard("
vip"))

    return
    if u["diamond"] <
info["cost_diamond"]:
        await
query.message.edit_text("❌
Diamond کافی نداری",
```

```

reply_markup=back_keyboard("
vip"))

        return

add_currency(user_id,
"diamond", -
info["cost_diamond"])
        set_field(user_id,
"vip", tier)
        unlocked =
check_achievements(user_id)
        text = f"🎉 تبریک! اکنون
VIP {tier} هستی."
        if unlocked:
            text += "\n\n🏆
دست‌آورد جدید: " + ",
".join(unlocked)
        await
query.message.edit_text(text
,
reply_markup=back_keyboard("
vip"))

```



```

# ----- اشتراک
# ----- ویژه با تلگرام استارز
# -----
elif data ==
"star_vip_menu":
    text = "🌟 <b>اشتراک‌های
(پرداخت با تلگرام استارز) LIBER ویژه
</b>\n\nیک اشتراک را انتخاب کن تا
:مزایا و قیمت‌هایش را ببینی"
    buttons = [

[InlineKeyboardButton(info["
title"],
callback_data=f"star_tier_{k
ey}")]]
        for key, info in
STAR_SUBSCRIPTIONS.items()
    ]

buttons.append([InlineKeyboa
rdButton("⬅️ BACK بازگشت",

```

```
callback_data="vip"))))
        await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=InlineKeyboardM
arkup(buttons))

        elif
data.startswith("star_tier_"
):
        tier_key =
data.split("_", 2)[2]
        plan =
STAR_SUBSCRIPTIONS.get(tier_
key)
        if not plan:
            await
query.message.edit_text("❌
اشتراک نامعتبر
",
reply_markup=back_keyboard("
star_vip_menu"))
            return
```

```

        text = (
            f"
{plan['title']}\n\n"
            f"🎁 مزایا:
{plan['benefits']}\n\n"
            "مدت اشتراک را انتخاب"
            "کن:"
        )
        buttons = [

[InlineKeyboardButton(f"
{days} روز - ★ {stars} استارز",
callback_data=f"star_buy_{ti
er_key}_{days}")]]
            for days, stars
in plan["durations"].items()
        ]

buttons.append([InlineKeyboa
rdButton("⬅️ بازگشت",
callback_data="star_vip_menu
")])

```

```
        await
query.message.edit_text(text
,
reply_markup=InlineKeyboardM
arkup(buttons))

        elif
data.startswith("star_buy_")
:
            _, _, tier_key,
days_str = data.split("_",
3)
            days = int(days_str)
            plan =
STAR_SUBSCRIPTIONS.get(tier_
key)
            if not plan or days
not in plan["durations"]:
                await
query.message.edit_text("❌
گزینه نامعتبر",
reply_markup=back_keyboard("
```

```
star_vip_menu"))
    return
    stars_price =
plan["durations"][days]
    await
context.bot.send_invoice(
    chat_id=user_id,
    title=f"
{plan['title']} – {days}
روزه",

description=f"مزایا:
{plan['benefits']}",
    payload=f"vip:
{tier_key}:{days}",
    currency="XTR",
    prices=
[LabeledPrice(f"
{plan['title']} ({days}
روز)", stars_price)],

provider_token="",
```

```

        )
        await
        query.answer("★ با پرداخت
        فاکتور
        استارز برای ارسال شد
        ,
        show_alert=True)

        elif data ==
        "star_liber_menu":
            text = "✳ <b>خرید
            LIBER استارز تلگرام
            قیمت‌ها</b>\n\n
            منصفانه و ثابت هستند.
            یک بسته انتخاب
            کن:"

            buttons = [

                [InlineKeyboardButton(f"
                {pack['title']} –
                {pack['liber']} LIBER – ★
                {pack['stars']}",
                callback_data=f"star_liber_b
                uy_{key}")]]

                for key, pack in
                STAR_LIBER_PACKS.items()

```

```
]
```

```
buttons.append([InlineKeyboardButton("⬅️ بازگشت",  
callback_data="shop")])
```

```
    await
```

```
query.message.edit_text(text  
, parse_mode=ParseMode.HTML,  
reply_markup=InlineKeyboardMarkup(buttons))
```

```
    elif
```

```
data.startswith("star_liber_  
buy_"):
```

```
        pack_key =
```

```
data.split("star_liber_buy_"  
, 1)[1]
```

```
        pack =
```

```
STAR_LIBER_PACKS.get(pack_  
key)
```

```
        if not pack:
```

```
            await
```

```
query.message.edit_text("❌  
بسته نامعتبر",  
reply_markup=back_keyboard("star_liber_menu"))  
    return  
    await  
context.bot.send_invoice(  
    chat_id=user_id,  
  
title=pack["title"],  
    description=f"  
{pack['liber']} LIBER مستقیم به  
کیف پول شما اضافه می‌شود",  
    payload=f"liber:  
{pack_key}",  
    currency="XTR",  
    prices=  
[LabeledPrice(pack["title"],  
pack["stars"])],  
  
provider_token="",  
    )
```



```

        await
query.answer("★ با پرداخت
استارز برای ارسال شد
show_alert=True)

# ----- اتحاد -
-----

elif data == "alliance":
    if u["alliance_id"]:
        conn = db()
        c =
conn.cursor()

c.execute("SELECT * FROM
alliances WHERE
alliance_id=?",
(u["alliance_id"],))
        alliance =
c.fetchone()
        conn.close()
        text = (
            f"🤝 <b>اتحاد"

```

```

{alliance['name']}</b>\n\n"
        f"💰 خزانہ:
{alliance['treasury']}
Coin\n"

        f"👑 رہبر:
{alliance['leader_id']}"
    )
    kb =
    InlineKeyboardMarkup([

    [InlineKeyboardButton("💰 کمک
    بہ خزانہ",
    callback_data="alliance_donate")],

    [InlineKeyboardButton("⬅️
    BACK
    بازگشت بہ منو",
    callback_data="back_main")],
    ])
    await
    query.message.edit_text(text
    , parse_mode=ParseMode.HTML,

```

```
reply_markup=kb)
    else:
        text = "🤝 هنوز  
عضو هیچ اتحادی نیستی"
        kb =
        InlineKeyboardMarkup([

        [InlineKeyboardButton("🏛️  
ساخت اتحاد جدید",
        callback_data="alliance_create")],

        [InlineKeyboardButton("⬅️  
BACK  
بازگشت به منو",
        callback_data="back_main")],
        ])
        await
        query.message.edit_text(text
        , reply_markup=kb)

        elif data ==
        "alliance_create":
```

```

conn = db()
c = conn.cursor()
c.execute(
    "INSERT INTO
alliances (name, leader_id,
treasury) VALUES (?, ?, 0)",
    (f"اتحاد"
{u['first_name']}",
user_id),
    )
    alliance_id =
c.lastrowid
    conn.commit()
    conn.close()
    set_field(user_id,
"alliance_id", alliance_id)
    unlocked =
check_achievements(user_id)
    text = "اتحاد جدید 🎉"
    "ساخته شد و رهبر آن شدی"
    if unlocked:
        text += "\n\n🏅"

```

```
دستاورد جديد: " + ",
".join(unlocked)
    await
query.message.edit_text(text
,
reply_markup=back_keyboard("
alliance"))

    elif data ==
"alliance_donate":
        amount = min(100,
u["coin"])
        if amount <= 0:
            await
query.message.edit_text("❌
Coin کافی نداري.",
reply_markup=back_keyboard("
alliance"))
        return

add_currency(user_id,
"coin", -amount)
```

```

        conn = db()
        c = conn.cursor()
        c.execute("UPDATE
alliances SET treasury =
treasury + ? WHERE
alliance_id=?", (amount,
u["alliance_id"]))
        conn.commit()
        conn.close()
        await
query.message.edit_text(f"✅
{amount} Coin به خزانه اتحاد اضافه
شد.",
reply_markup=back_keyboard("
alliance"))

# ----- دعوت -----
-----

elif data == "invite":
    bot_username =
context.bot.username
    link =

```

```

f"https://t.me/{bot_username}
}?start={user_id}"
    text = (
        "👤 <b>دعوت</b>
دوستان</b>\n\n"
        f"🔗 لینک اختصاصی
شما:\n{link}\n\n"
        f"👤 :تعداد زیرمجموعه
{u['ref_count']} نفر\n"
        "🎁 هر دعوت جدید:
50 LIBER + 200 Coin"
    )
    await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=back_keyboard()
)

# ----- جایزه
روزانه -----
elif data == "daily":
    today =

```

```
datetime.now().strftime("%Y-%m-%d")
        if
u["last_daily_reward"] ==
today:
            await
query.message.edit_text("✅
جایزه امروزت را قبلاً گرفتی
reply_markup=back_keyboard()
)
            return
            reward_liber =
random.randint(3, 15)
            set_field(user_id,
"last_daily_reward", today)

add_currency(user_id,
"liber", reward_liber)
            await
query.message.edit_text(
            f"🎁 جایزه روزانه
دریافت شد: <b>{reward_liber}</b>")
```



```

LIBER</b>",

parse_mode=ParseMode.HTML,

reply_markup=back_keyboard(
    ,
    )

    # ----- فروشگاه
    -----
    elif data == "shop":
        buttons = []
        row = []
        for key, item in
SHOP_ITEMS.items():

row.append(InlineKeyboardBut
ton(item["title"],
callback_data=f"shop_{key}")
)

        if len(row) ==
2:

```

```
buttons.append(row)
            row = []
        if row:

buttons.append(row)

buttons.append([InlineKeyboardButton("✳️ خرید LIBER با استارز",
callback_data="star_liber_menu")])

buttons.append([InlineKeyboardButton("⬅️ بازگشت به منو",
callback_data="back_main")])
        await
query.message.edit_text("🏠 24 LIBER: فروشگاه",
reply_markup=InlineKeyboardMarkup(buttons))
```

```
elif
data.startswith("shop_"):
    key =
data.split("_", 1)[1]
    item =
SHOP_ITEMS.get(key)
    if not item:
        await
query.message.edit_text("❌
آیتم نامعتبر",
reply_markup=back_keyboard("
shop"))

    return
    for currency, cost
in item["cost"].items():
        if u[currency] <
cost:
            await
query.message.edit_text(f"❌
{currency} کافی نداری",
reply_markup=back_keyboard("
shop"))
```

```

        return
    for currency, cost
in item["cost"].items():

    add_currency(user_id,
currency, -cost)
        field, amount =
item["give"]
        if field == "frame":

set_field(user_id, "frame",
amount)

            await
query.message.edit_text(f"✅
خرید موفق: {item['title']}",
reply_markup=back_keyboard("
shop"))
        else:

add_currency(user_id, field,
amount)

            await

```

```
query.message.edit_text(f"✅  
خرید موفق: {item['title']}",  
reply_markup=back_keyboard("shop"))
```

```
# -----  
دستاوردها -----  
elif data ==  
"achievements":  
    unlocked =  
    check_achievements(user_id)  
    unlocked_keys =  
    get_achievements(user_id)  
    lines = []  
    for key, ach in  
    ACHIEVEMENTS.items():  
        mark = "✅" if  
key in unlocked_keys else  
"🔒"  
        lines.append(f"  
{mark} {ach['title']} –  
{ach['desc']}")
```

```

        text = "🏆
<b>دستاوردهای شما</b>\n\n" +
"\n".join(lines)
        if unlocked:
            text += "\n\n🎉
دستورد جدید باز شد
".join(unlocked)
        await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=back_keyboard()
)

# ----- تحقیقات
-----
elif data == "research":
    info =
get_research_info(user_id)
    if info:
        text = (
            "🔬
<b>تحقیقات</b>\n\n"

```

```

        f"سطح فعلی:
        {u['research_level']}\n"
        f"تحقیق بعدی:
        {info['name']}\n"
        f"هزینه:
        {info['cost_coin']} Coin\n"
        f"اثر:
        {info['effect']}"
    )
    kb =
    InlineKeyboardMarkup([

    [InlineKeyboardButton("🔬
    ارتقا",
    callback_data="research_upgr
    ade")],

    [InlineKeyboardButton("⬅️
    BACK
    بازگشت به منو",
    callback_data="back_main")],

    ])
    else:

```

```
        text = "🔬 تمام  
سطوح تحقیقاتی را کامل کرده‌ای 🎉"  
        kb =  
back_keyboard()  
        await  
query.message.edit_text(text  
, parse_mode=ParseMode.HTML,  
reply_markup=kb)  
  
        elif data ==  
"research_upgrade":  
            success, msg =  
upgrade_research(user_id)  
            await  
query.message.edit_text(msg,  
reply_markup=back_keyboard("  
research"))  
  
        # ----- دفاع -----  
-----  
        elif data == "defense":  
            cost =
```



```

get_defense_upgrade_cost(u["
defense_level"])
    text = (
        "🛡️ <b>دفاع
کشور</b>\n\n"
        f"سطح فعلی دفاع:
{u['defense_level']}\n"
        f"هزینه ارتقای بعدی:
{cost} Coin"
    )
    kb =
InlineKeyboardMarkup([

[InlineKeyboardButton("🛡️
ارتقای دفاع",
callback_data="defense_upgra
de")],

[InlineKeyboardButton("⬅️
BACK",
callback_data="back_main")],
    ])

```

```

        await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=kb)

        elif data ==
"defense_upgrade":
            success, msg =
upgrade_defense(user_id)
            await
query.message.edit_text(msg,
reply_markup=back_keyboard("
defense"))

        # ----- اكتشاف
-----

        elif data ==
"exploration":
            text = (
                "
<b>اكتشاف</b>\n\n"
                f"حداقل سطح لازم:"

```

```

{EXPLORATION_MIN_LEVEL}\n"
        f"هزینه انرژی:"
{EXPLORATION_ENERGY_COST}\n"
        f"سطح فعلی شما:"
{u['level']} | انرژی:
{u['energy']}
    )
    kb =
InlineKeyboardMarkup([

[InlineKeyboardButton("🌌
شروع اکتشاف",
callback_data="exploration_g
o")],

[InlineKeyboardButton("⬅️
بازگشت به منو",
callback_data="back_main")],
    ])
    await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,

```

```

reply_markup=kb)

        elif data ==
"exploration_go":
            success, msg =
do_exploration(user_id)
            await
query.message.edit_text(msg,
reply_markup=back_keyboard("
exploration"))

        # ----- بازار
سیاه -----
        elif data ==
"black_market":
            item =
get_black_market_today()
            cost_text = ",
".join(f"{c} {k}" for k, c
in item["cost"].items())
            text = (
                "🕵️ <b>سیاه بازار

```

```

امروز</b>\n\n"
        f"
{item['title']}\n"
        f"💰 قیمت:
{cost_text}\n"
        "⌚ این پیشنهاد فردا
        تغییر می‌کند
    )
    kb =
    InlineKeyboardMarkup([

    [InlineKeyboardButton("🛒
خرید",
callback_data="black_market_
buy")],

    [InlineKeyboardButton("⬅️
BACK
بازگشت به منو",
callback_data="back_main")],
    ])
    await
    query.message.edit_text(text

```

```
, parse_mode=ParseMode.HTML,  
reply_markup=kb)
```

```
    elif data ==  
"black_market_buy":  
        success, msg =  
buy_black_market_item(user_i  
d)  
        await  
query.message.edit_text(msg,  
reply_markup=back_keyboard("  
black_market"))
```

```
    # ----- فصل -  
-----  
    elif data == "season":  
        maybe_reset_season()  
        number, days_left =  
get_season_info()  
        text = (  
            "📅 <b>فصل  
بازی</b>\n\n"
```

```

        f"فصل فعلی:
{number}\n"
        f"روزهای باقی مانده:
{days_left}\n"
        "🏆 در پایان هر فصل،
جوایز و مدال به برترین بازیکنان تعلق
می‌گیرد."
    )
    await
    query.message.edit_text(text
    , parse_mode=ParseMode.HTML,
    reply_markup=back_keyboard()
    )

    # ----- بازار
    (P2P) -----
    elif data ==
    "p2p_market":
        trades =
    list_open_trades()
        lines = [
            f"#

```

```

{t['trade_id']} –
{t['item_amount']}
{t['item_field']} به
{t['price_coin']} Coin"
        for t in trades
            ] or ["هیچ پیشنهادی"]
            ].موجود نیست
            text = "📦 <b>بازار
            مستقیم بازیکنان</b>\n\n" +
            "\n".join(lines)
            kb =
            InlineKeyboardMarkup([

            [InlineKeyboardButton("➕ ثبت
            LIBER",
            callback_data="p2p_sell_offe
            r")],

            [InlineKeyboardButton("⬅️
            BACK",
            callback_data="back_main")],
            ])

```



```
        await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=kb)

        elif data ==
        "p2p_sell_offer":
            # پیشنهاد پیش فرض نمونه:
            به قیمت بازار فعلی LIBER فروش ۲۰
            ضربدر ۲۰

            amount = 20
            price =
            round(get_market_price() *
            amount * 1.05, 2)

            success, msg =
            create_trade_offer(user_id,
            "liber", amount, price)

            await
            query.message.edit_text(msg,
            reply_markup=back_keyboard("
            p2p_market"))
```

```

elif
data.startswith("p2p_buy_"):
    trade_id =
int(data.split("_")[-1])
    success, msg =
accept_trade_offer(user_id,
trade_id)
    await
query.message.edit_text(msg,
reply_markup=back_keyboard("
p2p_market"))

# ----- بازار
پیش بینی قیمت -----
elif data ==
"prediction":
    price =
get_market_price()
    text = (
        "  <b>بازار پیش بینی</b>
        قیمت LIBER</b>\n\n"
        f"  قیمت فعلی:"

```

```

{price} Coin\n"
        f"$💰 شرط :
{PREDICTION_BET_AMOUNT}
Coin\n"
        f"🏆 برد:
ضرب
{PREDICTION_WIN_MULTIPLIER}x
\n\n"
        پیش بینی می کنی قیمت "
        "تا ساعت بعد بالا برود یا پایین بیاید؟
    )
    kb =
    InlineKeyboardMarkup([

    [InlineKeyboardButton("$📈",
    "صعودی",
    callback_data="predict_up"),

    InlineKeyboardButton("$📉",
    "نزولی",
    callback_data="predict_down"
    )],

```

```
[InlineKeyboardButton("←  
BACK",  
    بازگشت به منو",  
    callback_data="back_main")],  
    ])  
    await  
    query.message.edit_text(text  
    , parse_mode=ParseMode.HTML,  
    reply_markup=kb)  
  
    elif data in  
    ("predict_up",  
    "predict_down"):  
        direction = "up" if  
data == "predict_up" else  
"down"  
        success, msg =  
place_prediction(user_id,  
direction)  
        await  
query.message.edit_text(msg,  
reply_markup=back_keyboard("
```

```

prediction"))

# ----- رقابت
آنلاین -----
elif data ==
"competition":
    comp_league =
get_league_tier_name(u["rank
_points"])
    days_left =
get_tournament_info()
    text = (
        "× <b>آنلاین رقابت
LIBER</b>\n\n"
        f"× لیگ رقابتی فعلی:
{comp_league}\n"
        f"📊 امتیاز رنک:
{u['rank_points']}\n"
        f"🎮 بازی‌ها:
{u['matches_played']} | 🏆
برد: {u['matches_won']}\n"
        f"🏆 تا پایان تورنمنت"

```

```
{days_left} روز\n"
    f"🏆 ۵) جوایز تورنمنت
    "
    f"اول
    {TOURNAMENT_REWARDS[1]}
    دوم، LIBER
    سوم، {TOURNAMENT_REWARDS[2]}
    "
    f"چهارم {TOURNAMENT_REWARDS[3]}
    {TOURNAMENT_MEDAL_REWARDS[4]}
    پنجم، {TOURNAMENT_MEDAL_REWARDS[5]}
    "
    "\n\nمدال"
    "یک ورزش را انتخاب"
    "کن:"
    )
    kb =
    InlineKeyboardMarkup([
    [InlineKeyboardButton("⚽
    , "فوتبال",
    callback_data="sport_footbal
```

```
l"),
```

```
InlineKeyboardButton("🏀  
بسکتبال",  
callback_data="sport_basketb  
all")],
```

```
[InlineKeyboardButton("🏆  
رتبه بندی رقابتی",  
callback_data="competition_l  
eaderboard"),
```

```
InlineKeyboardButton("🏟️  
مسابقات اخیر",  
callback_data="competition_r  
ecent")],
```

```
[InlineKeyboardButton("❓  
راهنمای رقابت",  
callback_data="competition_h  
elp")],
```

```

[InlineKeyboardButton("⬅️ BACK",
    بازگشت به منو",
    callback_data="back_main")],
    ])
    await
    query.message.edit_text(text
    , parse_mode=ParseMode.HTML,
    reply_markup=kb)

    elif data ==
    "competition_leaderboard":
        rows =
        get_competition_leaderboard(
        )

        if not rows:
            text = "🏆"
            <b>رتبه‌بندی رقابتی</b>\n\nهنوز کسی
            !امتیاز رنگ نگرفته - اولین نفر باش
        else:
            lines = [
                f"{i+1}.
                {r['first_name']} -

```



```

        {r['rank_points']} امتیاز
        ({get_league_tier_name(r['rank_points'])}) | "
            f"
        {r['matches_won']}/{r['matches_played']} برد"
            for i, r in
enumerate(rows)
        ]
        text = "🏆
<b>رتبه‌بندی رقابتی برتر</b>\n\n" +
"\n".join(lines)
        await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=back_keyboard("
competition"))

        elif data ==
"competition_recent":
            rows =
get_recent_matches()

```

```

        if not rows:
            text = "🏆
<b>مسابقات اخیر</b>\n\nهنوز هیچ
مسابقه‌ای برگزار نشده
        else:
            lines = []
            for r in rows:
                opp_name =
r["opponent_name"] if
r["opponent_id"] != 0 else
"🤖 هوش مصنوعی"

            lines.append(
                f"⚔️
{SPORTS[r['sport']]
['title']} -
{r['player_name']}
{r['player_score']} -
{r['opponent_score']}
{opp_name}"
            )
            text = "🏆

```

```
<b>مسابقات اخير</b>\n\n" +
"\n".join(lines)
    await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=back_keyboard("
competition"))

    elif data ==
"competition_help":
        text = (
            "<b>راهنمای رقابت</b> ?"
            "<b>آنلاین</b>\n\n"
            "یک ورزش (فوتبال 1"
            "\n.یا بسکتبال) انتخاب کن"
            "2"
            "مهارت‌هایت را با"
            "ارتقا بده – هرچه مجموع قدرت Coin
            بالاتر باشد شانس بردت بیشتر
            \n.می‌شود"
            "3"
            "روی «شروع"
            "مسابقه رنک» بزن. اگر همان لحظه حریف
            واقعی منتظر باشد با او بازی می‌کنی؛ در
```

غیر این صورت بلافاصله با هوش مصنوعی
"\n". ربات مسابقه می‌دهی تا معطل نمانی
ورودی هر مسابقه 4" f

LIBER {MATCH_ENTRY_FEE} رنک
منهای کارمزد) است. برنده مجموع جایزه
{MATCH_POT_FEE_PERCENT}% را
"\n". می‌برد

در هر مسابقه 5" f
حمله رد و {MATCH_POSSESSIONS}
بدل می‌شود؛ در هر حمله قدرت حمله‌کننده
با قدرت دفاع مقایسه می‌شود - قدرت
بیشتر یعنی گل بیشتر. تساوی یعنی دفاع
"\n". در آخرین لحظه گل را گرفته

اگر نتیجه مساوی 6" "
"\n". شود، ضربات پنالتی تعیین‌کننده است
بردها امتیاز رنک 7" "

می‌دهند، باخت‌ها کم می‌کنند. امتیاز رنک
تعیین‌کننده لیگ رقابتی توست: مبتدی ←
حرفه‌ای ← استاد ← اژدهای آزاد ←
اژدهای افسانه‌ای ← اژدهای کامل افسانه‌ای
"\n". ← لیبر لجنده وان

هر ۲ ماه یکبار 8" "

تورنمت فصلی بسته می‌شود و ۵ نفر برتر
نقد، LIBER سه نفر اول) جایزه می‌گیرند
).\n"

f"🔥 9 «مسابقه»

:سخت» یک حالت پرمخاطره است

{HARD_MATCH_ENTRY_FEE} Coin

ورودی می‌دهی، حریف هوش مصنوعی

قوی‌تر از حد معمول شبیه‌سازی می‌شود،

{HARD_MATCH_REWARD} اما اگر ببری

Coin جایزه می‌گیری."

)

await

```
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=back_keyboard("
competition"))
```

elif

data.startswith("sport_"):

 sport =

data.split("_", 1)[1]

 sport_info =

```

SPORTS[sport]
    stats_lines = [
        f"{label}: سطح"
        {u[f'{sport}_{key}']} for
key, label in
sport_info["stats"].items()
    ]
    total_power =
get_total_power(user_id,
sport)
    text = (
        f"
        {sport_info['title']} -
        <b>پنل مهارت‌ها</b>\n\n"
        +
        "\n".join(stats_lines)
        + f"\n\n🥤 مجموع
        قدرت: {total_power}"
    )
    stat_buttons = [

InlineKeyboardButton(f"⬆️

```

```

{label}",
callback_data=f"upgrade_{sport}_{key}")
        for key, label
in
sport_info["stats"].items()
    ]
    rows =
[stat_buttons[i:i+2] for i
in range(0,
len(stat_buttons), 2)]

rows.append([InlineKeyboardB
utton("🎮 شروع مسابقه رنک",
callback_data=f"match_start_
{sport}")]])

rows.append([InlineKeyboardB
utton("🔥 مسابقه سخت
(پرمخاطره)",
callback_data=f"match_hard_{
sport}")]])

```

```
rows.append([InlineKeyboardB
utton("⬅️ بازگشت",
callback_data="competition")
])

        await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=InlineKeyboardM
arkup(rows))

        elif
data.startswith("upgrade_"):
        _, sport, stat_key =
data.split("_", 2)
        success, msg =
upgrade_stat(user_id, sport,
stat_key)
        await
query.message.edit_text(msg,
reply_markup=back_keyboard(f
"sport_{sport}"))
```



```

        elif
data.startswith("match_start
_"):
            sport =
data.split("_", 2)[2]
            if u["liber"] <
MATCH_ENTRY_FEE:
                await
query.message.edit_text(
                    f"❌ برای شرکت
در مسابقه به {MATCH_ENTRY_FEE}
LIBER نیاز داری.",

reply_markup=back_keyboard(f
"sport_{sport}"),
)
        return

add_currency(user_id,
"liber", -MATCH_ENTRY_FEE)

```

```
status, opponent_id  
= join_match_queue(user_id,  
sport)
```

```
if status ==  
"waiting":
```

```
    # حریف واقعی آنلاین  
    نبود؛ برای اینکه کاربر معطل نماند بلافاصله  
    با هوش مصنوعی مسابقه می‌دهد
```

```
leave_match_queue(user_id)  
    match_data =  
simulate_match(user_id,  
None, sport, vs_bot=True)
```

```
apply_match_result(user_id,  
0, sport, match_data,  
vs_bot=True)
```

```
    result_emoji =  
    "🏆 بردی" if  
match_data["result"] ==
```

```

"win" else "😓 باختی."
        text = (
            f"❌ <b>نتیجه
مسابقه {SPORTS[sport]
['title']}> (هوش مصنوعی)
</b>\n"

            "حریف واقعی"
            آنلاین نبود، مسابقه با ربات برگزار
            شد:\n\n"

            +
            "\n".join(match_data["log"])
            + f"\n\n📊

نتیجه نهایی: شما
{match_data['player_score']}

—
{match_data['opponent_score'
]}
{match_data['opponent_name'
]}\n"

            + f"🔋 قدرت
شما:
{match_data['player_power']}

```

```

| قدرت حریف:
{match_data['opponent_power'
]}\n\n"

+

result_emoji
)
await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=back_keyboard(f
"sport_{sport}"))
return

```

```

# حریف واقعی پیدا شد
(حریف قبلاً هزینه ورودی خودش را هنگام
ورود به صف پرداخت کرده است)
match_data =
simulate_match(user_id,
opponent_id, sport,
vs_bot=False)

apply_match_result(user_id,

```

```

opponent_id, sport,
match_data, vs_bot=False)

        result_emoji = "🏆
بردی!" if match_data["result"]
== "win" else "😞 باختی."
        text = (
            f"× <b>نتیجه مسابقه</b>
{SPORTS[sport]['title']}
</b>\n\n"
            +
            "\n".join(match_data["log"])
            + f"\n\n📊 نتیجه
نهایی: شما
{match_data['player_score']}
-
{match_data['opponent_score']}
{match_data['opponent_name']}
}\n"
            + f"🥤 قدرت شما:
{match_data['player_power']}

```

```

| قدرت حریف:
{match_data['opponent_power'
]}\n\n"
        + result_emoji
    )
    await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=back_keyboard(f
"sport_{sport}"))

    try:
        opp_result =
"win" if
match_data["result"] ==
"loss" else "loss"
        opp_emoji = "🏆"
        "بردی!" if opp_result == "win"
        else "😞 باختی."
        await
context.bot.send_message(
            opponent_id,

```

```

        f"❌ مسابقه
{SPORTS[sport]['title']} تمام
شد!\n"

        نتیجه:
        {match_data['opponent_score']
        ]} -
        {match_data['player_score']}
        \n{opp_emoji}",
        )
    except Exception:
        pass

    elif
data.startswith("match_hard_
"):
        sport =
data.split("_", 2)[2]
        if u["coin"] <
HARD_MATCH_ENTRY_FEE:
            await
query.message.edit_text(
        f"❌ برای مسابقه

```

```
سخت به {HARD_MATCH_ENTRY_FEE}  
Coin نیاز داری.",
```

```
reply_markup=back_keyboard(f  
"sport_{sport}"),  
)  
return
```

```
add_currency(user_id,  
"coin", -  
HARD_MATCH_ENTRY_FEE)  
match_data =  
simulate_hard_match(user_id,  
sport)
```

```
add_currency(user_id,  
"matches_played", 1)
```

```
if  
match_data["result"] ==  
"win":
```



```
add_currency(user_id,
"coin", HARD_MATCH_REWARD)

add_currency(user_id,
"matches_won", 1)

add_currency(user_id,
"rank_points",
RANK_WIN_POINTS)
        result_emoji =
f"🔥🏆 بردی!
{HARD_MATCH_REWARD} Coin جایزه
گرفتی!"
        else:
            u2 =
get_user(user_id)
            if
u2["rank_points"] +
RANK_LOSS_POINTS < 0:

set_field(user_id,
```

```

"rank_points", 0)
    else:

add_currency(user_id,
"rank_points",
RANK_LOSS_POINTS)
    result_emoji =
f"😞 باختی و
{HARD_MATCH_ENTRY_FEE} Coin
!را از دست دادی. حریف سخت بود

    text = (
        f"🔥 <b>مسابقه سخت
{SPORTS[sport]['title']}
</b>\n\n"
        +
        "\n".join(match_data["log"])
        + f"\n\n📊 نتیجه
نهایی: شما
{match_data['player_score']}
-
{match_data['opponent_score']

```

```

    ]}
    {match_data['opponent_name']}
    }\n"

    + f"🥤 قدرت شما:
    {match_data['player_power']}
    | قدرت حریف:
    {match_data['opponent_power'
    ]}\n\n"

    + result_emoji
    )
    await
    query.message.edit_text(text
    , parse_mode=ParseMode.HTML,
    reply_markup=back_keyboard(f
    "sport_{sport}"))

    elif data ==
    "match_cancel_queue":

    leave_match_queue(user_id)
    await
    query.message.edit_text("❌

```

```
    ", از صف انتظار خارج شدی",
    reply_markup=back_keyboard("
competition"))
```

```
        # ----- پنل
        ادمین -----
        elif data ==
        "admin_panel" and user_id in
        ADMIN_IDS:
            await
            show_admin_panel(query)

        elif data ==
        "admin_force_tournament" and
        user_id in ADMIN_IDS:
            await
            maybe_resolve_tournament_for
            ced(context.bot)
            await
            query.message.edit_text("✅
تورنمت فصلی برگزار شد و جوایز ارسال
شد.",
```

```
reply_markup=back_keyboard("
admin_panel"))
```

```
        elif data ==
"admin_info_broadcast" and
user_id in ADMIN_IDS:
            await
query.message.edit_text(
                "📢 برای ارسال پیام
همگانی از دستور زیر در چت استفاده
کن:\n<code>/broadcast متن
پیام</code>",

parse_mode=ParseMode.HTML,

reply_markup=back_keyboard("
admin_panel"),
        )

        elif data ==
"admin_info_ban" and user_id
in ADMIN_IDS:
```

```

        await
        query.message.edit_text(
            "🚫 برای مسدود یا رفع
            مسدودی کاربر از دستورات زیر استفاده
            کن:\n"
            "<code>/ban
            USER_ID</code>\n<code>/unban
            USER_ID</code>",

            parse_mode=ParseMode.HTML,

            reply_markup=back_keyboard("
            admin_panel"),
        )

        # ----- راهنما -----
        -----
        elif data == "help":
            text = (
                "❓ <b>راهنمای کامل</b>
                LIBER</b>\n\n"
                "🌐 <b>امپراتوری من</b>:"

```

کشورت رو بساز، مالیات جمع کن،
" \n" دفاع و فناوریات رو ارتقا بده

:بازار زنده "

 LIBER بخر و بفروش؛ قیمت هر
" \n" ساعت خودش تغییر می‌کنه

:بانک مرکزی "

 سپرده بذار سود بگیر، یا وام
" \n" بگیر

صندوق "

با :شانس

صندوق باز کن Coin/LIBER/Diamond
" \n" و جایزه شانسی بگیر

مأموریت "

و XP هر روز با یک کلیک :روزانه
" \n" رایگان بگیر LIBER

:رقابت آنلاین ✕"

 فوتبال یا بسکتبال انتخاب کن،
" \n" مهارت‌هاتو ارتقا بده و رنک‌بازی کن

با VIP: "

یا با تلگرام استارز اشتراک Diamond
ویژه بگیر و مزایای بیشتر داشته
" \n" باش

```

        خرید با<b> 🌟"
    از فروشگاه می‌تونی مستقیم </b>: استارز
    یا اشتراک LIBER با تلگرام استارز
    \n".\n"

    پشتیبانی<b> 📞"
    هر مشکلی داشتی از دکمه پشتیبانی </b>
    پیام بده، مستقیم برای ادمین ارسال
    \n\n".\n\n"

    هر سوالی داشتی از"
    ...! پشتیبانی بپرس"
    )
    await
    query.message.edit_text(text
    , parse_mode=ParseMode.HTML,
    reply_markup=back_keyboard()
    )

    # ----- پشتیبانی
    -----
    elif data == "support":

    context.user_data["awaiting_

```



```

support"] = True
    text = (
        "☎ <b>پشتیبانی</b>
LIBER</b>\n\n"
        "پیام خود را همینجا در"
        "چت تایپ و ارسال کن؛ پیام شما مستقیماً
        "برای ادمین ارسال می‌شود
        "و به‌زودی پاسخ داده"
        ".\n\n"
        "برای خرید اشتراک هم"
        "می‌تونی همینجا بنویسی، یا مستقیم از
        "اقدام کنی VIP ★ بخش
    )
    await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=back_keyboard()
)

# ----- مشاور
هوشمند -----
elif data ==

```

```

"smart_advisor":
    tips =
get_smart_advice(user_id)
    text = "🤖 <b>مشاور</b>
بر اساس LIBER</b>\n\n
وضعیت فعلیات، این پیشنهادات رو
دارم:\n\n" + "\n\n".join(
        f"• {tip}" for
tip in tips
    )
    kb =
InlineKeyboardMarkup([

[InlineKeyboardButton("🔄
تحلیل مجدد",
callback_data="smart_advisor
")],

[InlineKeyboardButton("⬅️
بازگشت به منو",
callback_data="back_main")],
    ])

```

```

        await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=kb)

        # ----- کد هدیه
        -----

        elif data ==
"gift_code":

context.user_data["awaiting_
gift_code"] = True
        text = (
            "🎁 <b>کد
هدیه</b>\n\n"
            "کد هدیهات را همینجا"
            "در چت تایپ و ارسال کن تا جایزه‌اش را"
            "دریافت کنی."
        )
        await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,

```

```

reply_markup=back_keyboard()
)

# ----- اخبار
جهانی -----
elif data == "news":
    news_lines =
get_world_news()
    text = "📰 <b>اخبار زنده</b>
LIBER</b>\n\n" +
"\n\n".join(f"• {line}" for
line in news_lines)
    kb =
InlineKeyboardMarkup([

[InlineKeyboardButton("🔄
بروزرسانی اخبار",
callback_data="news")],

[InlineKeyboardButton("⬅️
بازگشت به منو",
callback_data="back_main")],

```

```

    ])
    await
    query.message.edit_text(text
    , parse_mode=ParseMode.HTML,
    reply_markup=kb)

    # ----- برداشت
    TON -----
    elif data == "withdraw":
        my_requests =
        list_user_withdrawals(user_i
        d, limit=5)
        lines = [
            f"#
            {r['request_id']} -
            {r['amount_liber']} LIBER -
            {r['status']}"
            for r in
            my_requests
            ] or ["هیچ درخواستی ثبت
            نشده‌ای."]
        text = (

```

```

        برداشت<b> 
LIBER به TON</b>\n\n"
        موجودی فعلی :
        {u['liber']} LIBER\n"
        حداقل مبلغ :
        برداشت: {MIN_WITHDRAW_LIBER}
LIBER\n"

        کارمزد برداشت :
        {WITHDRAW_FEE_PERCENT}%\n\n"
        آخرین 
        +\n":درخواست‌های تو
        "\n".join(lines)
        )
        kb =
        InlineKeyboardMarkup([

        [InlineKeyboardButton("ثبت ,
        درخواست برداشت جدید",
        callback_data="withdraw_new"
        )],

        [InlineKeyboardButton("← BACK

```

```

        "بازگشت به منو",
        callback_data="back_main")],
    ])
    await
    query.message.edit_text(text
    , parse_mode=ParseMode.HTML,
    reply_markup=kb)

    elif data ==
    "withdraw_new":
        if u["liber"] <
        MIN_WITHDRAW_LIBER:
            await
            query.message.edit_text(
                f"❌ برای ثبت
                درخواست برداشت حداقل به
                {MIN_WITHDRAW_LIBER} LIBER
                نیاز داری.\n"
                موجودی فعلی"
                تو: {u['liber']} LIBER",
            reply_markup=back_keyboard("

```

```

withdraw"),
    )
    return

context.user_data["awaiting_
withdraw_amount"] = True
    await
query.message.edit_text(
    f"📁 چند LIBER
    \nمی‌خواهی برداشت کنی؟
    حداقل (
    {MIN_WITHDRAW_LIBER}، حداکثر
    {u['liber']})\n\n
    فقط عدد رو تایپ"
    ,
    reply_markup=back_keyboard("
withdraw"),
    )

# ----- واریز -
-----

```



```

elif data == "deposit":
    text = (
        "📥 <b>/ واریز</b>\n\n"
        "افزایش موجودی  

        در حال حاضر امکان  

        به صورت خودکار وجود TON واریز مستقیم  

        ندارد، ولی می‌تونی همین الان  

        با تلگرام استارز"  

        LIBER بخری و آنی به کیف پولت اضافه  

        بشه - سریع، امن و بدون نیاز به تایید  

        ادمین."  

    )
    kb =
    InlineKeyboardMarkup([

    [InlineKeyboardButton("🌟  

    با استارز LIBER خرید  

    callback_data="star_liber_me  

    nu")],

    [InlineKeyboardButton("⬅️  

    بازگشت به منو",

```

```
callback_data="back_main"]],
    ])
    await
query.message.edit_text(text
, parse_mode=ParseMode.HTML,
reply_markup=kb)

    else:
        await
query.message.edit_text("🔧
این بخش هنوز در حال توسعه است",
reply_markup=back_keyboard()
)

async def
build_admin_dashboard_text()
:
    now =
datetime.now().strftime("%Y-
%m-%d %H:%M:%S")
    conn = db()
```

```
c = conn.cursor()
c.execute("SELECT
COUNT(*) as cnt FROM users")
total_users =
c.fetchone()["cnt"]
c.execute("SELECT
COUNT(*) as cnt FROM users
WHERE banned=1")
banned_users =
c.fetchone()["cnt"]
c.execute("SELECT
COALESCE(SUM(liber),0) as s
FROM users")
total_liber =
c.fetchone()["s"]
c.execute("SELECT
COALESCE(SUM(coin),0) as s
FROM users")
total_coin =
c.fetchone()["s"]
c.execute("SELECT
COUNT(*) as cnt FROM
```

```
matches")
    total_matches =
c.fetchone()["cnt"]
    c.execute("SELECT
COUNT(*) as cnt FROM
match_queue")
    queue_count =
c.fetchone()["cnt"]
    c.execute("SELECT
first_name, rank_points FROM
users ORDER BY rank_points
DESC LIMIT 1")
    top_row = c.fetchone()
    c.execute("SELECT
COUNT(*) as cnt FROM users
WHERE warn_count > 0")
    warned_users =
c.fetchone()["cnt"]
    c.execute("SELECT
COUNT(*) as cnt FROM
withdrawal_requests WHERE
status='pending'")
```

```
pending_withdrawals =
c.fetchone()["cnt"]
conn.close()

days_left =
get_tournament_info()
top_competitor = f"
{top_row['first_name']}
({top_row['rank_points']}
امتیاز)" if top_row else "-"

text = (
    "👑 <b>پنل مدیریت</b>\n"
    "TITAN</b>\n"

    "—————\n"
    f"⌚ زمان سرور:
{now}\n"
    f"👤 کل کاربران:
{total_users}\n"
    f"🚫 کاربران مسدود:
{banned_users}\n"
```

```
f"⚠️ کاربران دارای اخطار:  
{warned_users}\n"
```

```
"_____\n"
```

```
f"📈 قیمت بازار:  
{get_market_price()} Coin\n"
```

```
f"👑 در مجموع LIBER  
اقتصاد:
```

```
{round(total_liber,2)}\n"
```

```
f"💰 در مجموع Coin  
اقتصاد:
```

```
{round(total_coin,2)}\n"
```

```
"_____\n"
```

```
f"🏆 مجموع مسابقات  
انجام شده: {total_matches}\n"
```

```
f"⌚ کاربران در صف انتظار  
مسابقه: {queue_count}\n"
```

```
f"🏅 برترین رقابت گر:  
{top_competitor}\n"
```

```
f"🏆 تا پایان تورنمنت فصلی:  
{days_left} روز\n"
```

```
        f"📁 برداشت درخواست‌های  
در انتظار: {pending_withdrawals}  
(/withdrawals)"  
    )  
    return text
```

```
async def  
show_admin_panel(query):  
    text = await  
build_admin_dashboard_text()  
    kb =  
InlineKeyboardMarkup([  
  
[InlineKeyboardButton("🔄  
بروزرسانی",  
callback_data="admin_panel")  
  
,  
  
InlineKeyboardButton("🏆  
برگزاری فوری تورنمنت",  
callback_data="admin_force_t
```

```
ournament"]],
```

```
[InlineKeyboardButton("📢 پیام  
همگانی (/broadcast)",  
callback_data="admin_info_br  
oadcast"),
```

```
InlineKeyboardButton("🚫  
مسدودسازی (/ban)",  
callback_data="admin_info_ba  
n")]],
```

```
[InlineKeyboardButton("⬅️  
BACK بازگشت به منو",  
callback_data="back_main")],  
])  
await
```

```
query.message.edit_text(text  
, parse_mode=ParseMode.HTML,  
reply_markup=kb)
```



```

#
=====
=====
=====
# دستورات ادمین (متنی)
#
=====
=====
=====

async def
admin_command(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    user_id =
update.effective_user.id
    if user_id not in
ADMIN_IDS:
        await
update.message.reply_text("
🚫 شما دسترسی ادمین ندارید")
    return

```

```
        text = await
build_admin_dashboard_text()
        kb =
InlineKeyboardMarkup([

[InlineKeyboardButton("🔄
بروزرسانی",
callback_data="admin_panel")
,

InlineKeyboardButton("🏆
برگزاری فوری تورنمنت",
callback_data="admin_force_t
ournament")],
        ])
        await
update.message.reply_text(te
xt,
parse_mode=ParseMode.HTML,
reply_markup=kb)
```

```

async def
ban_command(update: Update,
context:
ContextTypes.DEFAULT_TYPE):
    user_id =
update.effective_user.id
    if user_id not in
ADMIN_IDS:
        await
update.message.reply_text("
❌ شما دسترسی ادمین ندارید")
        return
    if not context.args:
        await
update.message.reply_text("اسم
تفاده: /ban USER_ID")
        return
    target_id =
int(context.args[0])
    set_field(target_id,
"banned", 1)
    await

```

```
update.message.reply_text(f"  
🚫 مسدود شد {target_id} کاربر")
```

```
async def  
unban_command(update:  
Update, context:  
ContextTypes.DEFAULT_TYPE):  
    user_id =  
update.effective_user.id  
    if user_id not in  
ADMIN_IDS:  
        await  
update.message.reply_text("  
🚫 شما دسترسی ادمین ندارید")  
        return  
    if not context.args:  
        await  
update.message.reply_text("اسم  
تفاده: /unban USER_ID")  
        return  
    target_id =
```

```
int(context.args[0])
    set_field(target_id,
    "banned", 0)
    await
update.message.reply_text(f"
✅ از مسدودی {target_id} کاربر
خارج شد")
```

```
async def
broadcast_command(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    user_id =
update.effective_user.id
    if user_id not in
ADMIN_IDS:
        await
update.message.reply_text("
❌ شما دسترسی ادمین ندارید")
    return
    if not context.args:
```

```

        await
update.message.reply_text("اسه
متن پیام /broadcast :تفاده")
        return
        message_text = "
".join(context.args)
        conn = db()
        c = conn.cursor()
        c.execute("SELECT
user_id FROM users")
        all_users = c.fetchall()
        conn.close()
        sent = 0
        for row in all_users:
            try:
                await
context.bot.send_message(row
["user_id"], f"📢
{message_text}")
                sent += 1
            except Exception:
                pass

```

```
        await
update.message.reply_text(f"
✅ کاربر ارسال {sent} پیام برای
شد.")
```

```
async def
setbio_command(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    user_id =
update.effective_user.id
    if is_banned(user_id):
        return
    if not context.args:
        await
update.message.reply_text("اسم
متن بیوگرافی /setbio :تفاده
عاشق اقتصاد و /setbio :مثال\nتو
فوتبال ⚽💰")
    return
    bio_text = "
```

```

".join(context.args)
    if
contains_banned_word(bio_text):
        await
update.message.reply_text("
⚠️ این متن شامل الفاظ نامناسب است و
ذخیره نشد.")
        return
        if len(bio_text) > 150:
            bio_text =
bio_text[:150]
            set_field(user_id,
"bio", bio_text)
            await
update.message.reply_text("
✅ بيوگرافي شما بروزرسانی شد. از منوی
«پروفايل من» می‌توانی آن را ببینی

async def
text_message_filter(update:

```



```

Update, context:
ContextTypes.DEFAULT_TYPE):
    """اول پیام پشتیبانی را بررسی
    می‌کند، سپس فیلتر فحش ساده را روی
    پیام‌های متنی کاربر اجرا می‌کند"""
    if not update.message or
    not update.message.text:
        return
    user_id =
update.effective_user.id
    if is_banned(user_id):
        return

    # ----- پیام
    پشتیبانی -----
    if
context.user_data.get("await
ing_support"):

context.user_data["awaiting_
support"] = False
    u =

```

```

get_user(user_id)
    username_display =
f"@{u['username']}" if u and
u["username"] else "بدون نام"
    کاربری"

    forward_text = (
        "📧 <b>پیام پشتیبانی</b>\n\n"
        جدید</b>\n\n"
        f"👤 نام:
{update.effective_user.first
_name}\n"
        f"🗨 نام کاربری:
{username_display}\n"
        f"🔑 ID: <code>
{user_id}</code>\n\n"
        f"📧 متن
پیام:\n{update.message.text}\n
\n"
        f"برای پاسخ:
<code>/reply {user_id} متن
پاسخ</code>"
    )

```

```
        sent_to_any = False
        for admin_id in
ADMIN_IDS:
            try:
                await
context.bot.send_message(adm
in_id, forward_text,
parse_mode=ParseMode.HTML)
                sent_to_any
= True
            except Exception
as e:

logger.warning(f"Could not
forward support message to
admin {admin_id}: {e}")
            if sent_to_any:
                await
update.message.reply_text("
✅ پیام شما برای پشتیبانی ارسال شد.
")
            else:
```

```
        await
        update.message.reply_text("
        ⚠️ در حال حاضر پشتیبانی در دسترس
        نیست، بعداً دوباره امتحان کن
        ")
        return

        # ----- کد هدیه -----
        -----
        if
        context.user_data.get("await
        ing_gift_code"):

        context.user_data["awaiting_
        gift_code"] = False
        success, msg =
        redeem_gift_code(user_id,
        update.message.text)
        await
        update.message.reply_text(ms
        g)

        return
```

```

# ----- برداشت
TON: (مبلغ) ۱ مرحله -----
-----
    if
context.user_data.get("await
ing_withdraw_amount"):

context.user_data["awaiting_
withdraw_amount"] = False
    u =
get_user(user_id)
    try:
        amount =
float(update.message.text.st
rip())
    except ValueError:
        await
update.message.reply_text("
❌ لطفاً فقط عدد وارد کن")
    return
    if amount <
MIN_WITHDRAW_LIBER:

```

```
        await
update.message.reply_text(f"
❌ حداقل مبلغ برداشت
{MIN_WITHDRAW_LIBER} LIBER
است.")

        return
    if amount >
u["liber"]:
        await
update.message.reply_text(f"
❌ موجودی کافی نداری. موجودی فعلی:
{u['liber']} LIBER")
        return

context.user_data["pending_w
ithdraw_amount"] = amount

context.user_data["awaiting_
withdraw_address"] = True
        await
update.message.reply_text("
📄 خودت رو TON حالا آدرس کیف پول
```

```

    ):"وارد کن
        return

        # ----- برداشت
TON: (آدرس) ۲ مرحله -----
-----
        if
context.user_data.get("await
ing_withdraw_address"):

context.user_data["awaiting_
withdraw_address"] = False
        amount =
context.user_data.pop("pendi
ng_withdraw_amount", None)
        if not amount:
            await
update.message.reply_text("
❌ خطا در ثبت درخواست، دوباره از
شروع کن.")
        return
        ton_address =

```

```

update.message.text.strip()
        success, result =
create_withdraw_request(user
_id, amount, ton_address)
        if not success:
            await
update.message.reply_text(re
sult)

        return
        request_id = result
        u =
get_user(user_id)
        is_suspicious =
check_suspicious_liber_gain(
user_id, amount)
        await
update.message.reply_text(
        f"✅ درخواست برداشت
#{request_id} ثبت شد و در انتظار
.\n"
        تایید ادمین است
        f"$💰 مبلغ:
{amount} LIBER\n📍 آدرس:

```



```

{ton_address}\n\n"
        به محض بررسی، نتیجه"
        ".برایت ارسال می‌شود
    )
    suspicious_line = (
        f"\n\n🚨 <b>هشدار:
        این مبلغ بیشتر از حد معمول است </b>
        ({SUSPICIOUS_LIBER_GAIN_THRESHOLD}+ LIBER) – قبل از تایید،
        بررسی /finduser حساب کاربر رو با
        کن."
        if is_suspicious
    else ""
    )
    forward_text = (
        "📧 <b>درخواست
        </b>\n\n"
        f"📄 شماره درخواست
        #{request_id}\n"
        f"👤 کاربر:
        {update.effective_user.first
        _name} (@{u['username']} or

```

```

    '-'})\n"
        f"👤 آیدی: <code>
{user_id}</code>\n"
        f"💰 مبلغ:
{amount} LIBER\n"
        f"📍 آدرس TON:
{ton_address}"
        +
        suspicious_line +
        f"\n\nبرای تایید:
<code>/approvewithdraw
{request_id}</code>\n"
        f"برای رد:
<code>/rejectwithdraw
{request_id}</code>"
    )
    for admin_id in
ADMIN_IDS:
        try:
            await
context.bot.send_message(adm
in_id, forward_text,

```

```

parse_mode=ParseMode.HTML)
        except Exception
as e:

logger.warning(f"Could not
notify admin {admin_id}
about withdrawal: {e}")
        return

        # ----- فیلتر
        فحش -----
        if
contains_banned_word(update.
message.text):
            conn = db()
            c = conn.cursor()
            c.execute("UPDATE
users SET
warn_count=warn_count+1
WHERE user_id=?",
(user_id,))
            conn.commit()

```

```
c.execute("SELECT
warn_count FROM users WHERE
user_id=?", (user_id,))
warn_count =
c.fetchone()["warn_count"]
conn.close()
if warn_count >=
MAX_WARN_BEFORE_BAN:

set_field(user_id, "banned",
1)

await
update.message.reply_text("
❌ به دلیل استفاده مکرر از الفاظ
نامناسب، حساب شما مسدود شد")
else:
    await
    update.message.reply_text(
        f"⚠️ لطفاً از
        الفاظ نامناسب استفاده نکن.
        {warn_count}/{MAX_WARN_BEFORE_BAN}")
```

)

```
async def
reply_command(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    """ادمین با این دستور به پیام
    /reply :پشتیبانی کاربر پاسخ می‌دهد
    USER_ID متن پاسخ"""
    admin_id =
update.effective_user.id
    if admin_id not in
ADMIN_IDS:
        await
update.message.reply_text("
❌ شما دسترسی ادمین ندارید")
        return
    if len(context.args) <
2:
        await
update.message.reply_text("اسم")
```

```

    متن پاسخ /reply USER_ID :تفاده
    return
    try:
        target_id =
int(context.args[0])
        except ValueError:
            await
update.message.reply_text("
❌.آیدی عددی نامعتبر است")
        return
        reply_text = "
".join(context.args[1:])
        try:
            await
context.bot.send_message(target_id, f"📩 <b>پاسخ پشتیبانی:
</b>\n\n{reply_text}",
parse_mode=ParseMode.HTML)
            await
update.message.reply_text("
✅.پاسخ ارسال شد")
        except Exception as e:

```

```
await
update.message.reply_text(f"
❌ ارسال پاسخ ناموفق بود {e}")
```

```
async def
finduser_command(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    """جزئیات کامل یک کاربر را نشان
    می‌دهد: /finduser USER_ID"""
    admin_id =
update.effective_user.id
    if admin_id not in
ADMIN_IDS:
        await
update.message.reply_text("
🚫 شما دسترسی ادمین ندارید")
        return
    if not context.args:
        await
update.message.reply_text("اسم")
```

```
تفاده: /finduser USER_ID")
    return
    try:
        target_id =
int(context.args[0])
    except ValueError:
        await
update.message.reply_text("
❌ آیدی عددی نامعتبر است")
        return
        u = get_user(target_id)
        if not u:
            await
update.message.reply_text("
❌ کاربری با این آیدی پیدا نشد")
        return
        text = (
            f"👤 <b>
{u['first_name']}</b>
(@{u['username'] or '-'})\n"
            f"🟢 <code>
{target_id}</code>\n"
```



```

        f"👉 LIBER:
        {u['liber']} | 💰 Coin:
        {u['coin']} | 💎 Diamond:
        {u['diamond']}\n"

        f"★ سطح:
        {u['level']} | VIP:
        {u['vip']} تا
        {u['vip_expires_at']} or
        '-'}\n"

        f"🚫 مسدود: {'بله' if
        u['banned'] else 'خير'} | ⚠️
        اخطار: {u['warn_count']}\n"

        f"📅 17 عضویت:
        {u['joined_at']} | 🔄 ورود:
        {u['login_count']}"
    )
    await
    update.message.reply_text(te
    xt,
    parse_mode=ParseMode.HTML)

```

```

async def
grantliber_command(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    """ LIBER تکمیل دستی خرید """
    /grantliber توسط ادمین
    USER_ID AMOUNT"""
    admin_id =
update.effective_user.id
    if admin_id not in
ADMIN_IDS:
        await
update.message.reply_text("
❌ شما دسترسی ادمین ندارید")
        return
    if len(context.args) <
2:
        await
update.message.reply_text("اسم
/grantliber USER_ID
مقدار")
    return

```

```
try:
    target_id =
int(context.args[0])
    amount =
float(context.args[1])
except ValueError:
    await
update.message.reply_text("
❌ مقدار یا آیدی نامعتبر است")
    return
    add_currency(target_id,
"liber", amount)
    await
update.message.reply_text(f"
✅ {amount} LIBER به کاربر
{target_id} اضافه شد.")
    try:
        await
context.bot.send_message(target_id, f"🎉 {amount} LIBER
توسط پشتیبانی به کیف پول شما اضافه
شد!")
```

```
except Exception:
    pass

async def
grantvip_command(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    """تکمیل دستی خرید اشتراک"""
    /grantvip USER_ID
    TIER DAYS
    TIER از یکی: normal,
    dragon, dragon_legend"""
    admin_id =
    update.effective_user.id
    if admin_id not in
    ADMIN_IDS:
        await
    update.message.reply_text("
    شما دسترسی ادمین ندارید")
    return
    if len(context.args) <
```

```

3:
    await
update.message.reply_text("اسم
تتفاده: /grantvip USER_ID TIER
DAYS\nTIER: normal | dragon
| dragon_legend")
    return
    try:
        target_id =
int(context.args[0])
        tier_key =
context.args[1]
        days =
int(context.args[2])
    except ValueError:
        await
update.message.reply_text("
❌ ورودی نامعتبر است")
    return
    if tier_key not in
STAR_SUBSCRIPTIONS:
        await

```

```

update.message.reply_text("
❌ (normal) نوع اشتراک نامعتبر است
| dragon | dragon_legend).")
    return
    plan =
STAR_SUBSCRIPTIONS[tier_key]
    start, end =
activate_star_subscription(t
arget_id, tier_key, days)
    await
update.message.reply_text(f"
✅ {plan['title']} برای
{days} روز به کاربر {target_id}
داده شد.")
    try:
        await
context.bot.send_message(
    target_id,
    f"🎉 <b>اشتراک شما</b>
: نوع 🏷️
{plan['title']}\n"
    f"📅 از 17:

```

```
{start.strftime('%Y-%m-%d')}  
تا {end.strftime('%Y-%m-%  
%d')}\n\n"
```

```
f"🎁 مزایا:  
{plan['benefits']}\n\nمبارک  
🐉! باشه، لذت ببرید
```

```
parse_mode=ParseMode.HTML,  
)  
except Exception:  
    pass
```

```
async def  
createcode_command(update:  
Update, context:  
ContextTypes.DEFAULT_TYPE):  
    """ساخت کد هدیه:  
/createcode CODE FIELD  
AMOUNT MAX_USES  
    FIELD از یکی: liber, coin,  
diamond, medal"""
```

```
        admin_id =
update.effective_user.id
        if admin_id not in
ADMIN_IDS:
            await
update.message.reply_text("
❌ شما دسترسی ادمین ندارید.")
            return
        if len(context.args) <
4:
            await
update.message.reply_text("اسم: /createcode CODE FIELD
تفاده: AMOUNT MAX_USES\nFIELD:
liber | coin | diamond |
medal")
            return
        code, field, amount_str,
max_uses_str =
context.args[0],
context.args[1],
context.args[2],
```



```
context.args[3]
    if field not in
("liber", "coin", "diamond",
"medal"):
        await
update.message.reply_text("
❌ نوع جایزه نامعتبر است (liber |
coin | diamond | medal).")
        return
    try:
        amount =
float(amount_str)
        max_uses =
int(max_uses_str)
        except ValueError:
            await
update.message.reply_text("
❌ مقدار یا تعداد استفاده نامعتبر
است.")
            return
        success =
create_gift_code(code,
```

```
field, amount, max_uses,
admin_id)
    if success:
        await
update.message.reply_text(f"
✅ کد هدیه «{code.upper()}»
تا {amount} {field} ساخته شد
({max_uses} نفر).")
    else:
        await
update.message.reply_text("
❌ این کد قبلاً ساخته شده - یک کد
دیگر انتخاب کن")
```

```
async def
withdrawals_command(update:
Update, context:
ContextTypes.DEFAULT_TYPE):
    """نمایش لیست درخواست‌های
    برداشت در انتظار تایید"""
    admin_id =
```

```
update.effective_user.id
    if admin_id not in
ADMIN_IDS:
        await
update.message.reply_text("
❌ شما دسترسی ادمین ندارید")
        return
        pending =
list_pending_withdrawals()
        if not pending:
            await
update.message.reply_text("
📅 هیچ درخواست برداشتی در انتظار
نیست.")
            return
            lines = []
            for r in pending:
                u =
get_user(r["user_id"])
                lines.append(
                    f"#
{r['request_id']} -
```

```

{u['first_name'] if u else
'-'} ({r['user_id']}) - "
        f"
{r['amount_liber']] LIBER →
{r['ton_address']}]}"
    )
    text = "📄 <b>درخواست‌های</b>\n\n" +
    "\n".join(lines) + (
        "\n\n تایید:
/withdraw ID\n برای رد:
/reject ID"
    )
    await
update.message.reply_text(te
xt,
parse_mode=ParseMode.HTML)

async def
withdraw_command(upda
te: Update, context:

```

```

ContextTypes.DEFAULT_TYPE):
    """تایید درخواست برداشت:
    /approvewithdraw
    REQUEST_ID"""
    admin_id =
    update.effective_user.id
    if admin_id not in
    ADMIN_IDS:
        await
    update.message.reply_text("
    ❌ شما دسترسی ادمین ندارید")
    return
    if not context.args:
        await
    update.message.reply_text("اسم
    استفاده: /approvewithdraw
    REQUEST_ID")
    return
    try:
        request_id =
    int(context.args[0])
    except ValueError:

```

```
        await
        update.message.reply_text("
❌ شماره درخواست نامعتبر است")
        return
        success, req =
approve_withdraw_request(req
uest_id)
        if not success:
            await
            update.message.reply_text("
❌ این درخواست پیدا نشد یا قبلاً
پرداش شده")
            return
            net_amount =
round(req["amount_liber"] -
req["fee_liber"], 2)
            await
            update.message.reply_text(f"
✅ تایید #{request_id} درخواست
{net_amount} : شد. مبلغ خالص
LIBER برای واریز دستی TON معادل")
            try:
```

```

        await
context.bot.send_message(
    req["user_id"],
    f"✅ درخواست برداشت
#{request_id} شد! تو تایید شد
    f"💰 (بعد مبلغ خالص
    {WITHDRAW_FEE_PERCENT}%
    از {net_amount} LIBER
    (کارمزد): {net_amount} LIBER
    معادل TON\n"
    f"📍 به آدرس:
    {req['ton_address']}\n\nبه زودی
    TON, به کیف پولت واریز می‌شه
    )
    except Exception:
        pass

```

```

async def
rejectwithdraw_command(updat
e: Update, context:
ContextTypes.DEFAULT_TYPE):
    """رد درخواست برداشت و

```

```
    بازگرداندن مبلغ: /rejectwithdraw
    REQUEST_ID""
    admin_id =
    update.effective_user.id
    if admin_id not in
    ADMIN_IDS:
        await
    update.message.reply_text("
    شما دسترسی ادمین ندارید")
    return
    if not context.args:
        await
    update.message.reply_text("اسم
    استفاده: /rejectwithdraw
    REQUEST_ID")
    return
    try:
        request_id =
    int(context.args[0])
    except ValueError:
        await
    update.message.reply_text("
    ")
```



```
)".شماره درخواست نامعتبر است ❌
    return
    success, req =
reject_withdraw_request(request_id)
    if not success:
        await
update.message.reply_text("
❌ این درخواست پیدا نشد یا قبلاً
).پردازش شده
    return
    await
update.message.reply_text(f"
🔄 رد شد #{request_id} درخواست
و {req['amount_liber']} LIBER
).به کاربر بازگردانده شد
    try:
        await
context.bot.send_message(
            req["user_id"],
            f"❌ درخواست برداشت
#{request_id} تو رد شد و
```

```
{req['amount_liber']} LIBER
به کیف پولت برگشت
    برای اطلاع بیشتر با"
    , "پشتیبانی صحبت کن
    )
except Exception:
    pass
```

```
#
=====
=====
=====
# پرداخت با تلگرام استارز (Telegram
Stars)
#
=====
=====
=====
```

```
async def
precheckout_callback(update:
```

```
Update, context:
ContextTypes.DEFAULT_TYPE):
    query =
update.pre_checkout_query
    await
query.answer(ok=True)

async def
successful_payment_callback(
update: Update, context:
ContextTypes.DEFAULT_TYPE):
    payment =
update.message.successful_pa
yment
    user_id =
update.effective_user.id
    payload =
payment.invoice_payload

    if
payload.startswith("vip:"):

```

```
_, tier_key,
days_str =
payload.split(":")
days = int(days_str)
plan =
STAR_SUBSCRIPTIONS.get(tier_
key)
start, end =
activate_star_subscription(u
ser_id, tier_key, days)
title =
plan["title"] if plan else
tier_key
text = (
f"🎉 <b>اشتراک شما</b>\n\n"
f"🏷️ نوع: {title}\n"
f"📅 از تاریخ {start.strftime('%Y-%m-%d')}\n"
f"📅 تا تاریخ {end.strftime('%Y-%m-%d')}
```

```

{end.strftime('%Y-%m-%d')}\n\n"

        f"🎁 مزایا:
        {plan['benefits']} if plan
        else ''}\n\n"

        "مبارک باشه، لذت"
        "🐍! ببرید"
    )
    await
    update.message.reply_text(text,
    parse_mode=ParseMode.HTML)

    elif
    payload.startswith("liber:")
    :
        pack_key =
        payload.split(":", 1)[1]
        amount =
        grant_liber_pack(user_id,
        pack_key)
        await

```

```

update.message.reply_text(
    f"✅ خرید موفق!
    به کیف پول شما LIBER {amount}
    اضافه شد. لذت ببرید 🙏"
)

else:

logger.warning(f"Unknown
payment payload: {payload}")

#
=====
=====
=====
# main
#
=====
=====
=====

```

```
def main():  
    init_db()  
    app =  
    Application.builder().token(  
        BOT_TOKEN).build()  
  
    app.add_handler(CommandHandler(  
        "start", start))  
  
    app.add_handler(CommandHandler(  
        "admin", admin_command))  
  
    app.add_handler(CommandHandler(  
        "ban", ban_command))  
  
    app.add_handler(CommandHandler(  
        "unban", unban_command))  
  
    app.add_handler(CommandHandler(  
        "broadcast",  
        broadcast_command))
```

```
app.add_handler(CommandHandler(
    "setbio",
    setbio_command))
```

```
app.add_handler(CommandHandler(
    "reply", reply_command))
```

```
app.add_handler(CommandHandler(
    "finduser",
    finduser_command))
```

```
app.add_handler(CommandHandler(
    "grantliber",
    grantliber_command))
```

```
app.add_handler(CommandHandler(
    "grantvip",
    grantvip_command))
```

```
app.add_handler(CommandHandler(
    "createcode",
```



```
createcode_command))
```

```
app.add_handler(CommandHandler("withdrawals",  
withdrawals_command))
```

```
app.add_handler(CommandHandler("approvewithdraw",  
approvewithdraw_command))
```

```
app.add_handler(CommandHandler("rejectwithdraw",  
rejectwithdraw_command))
```

```
app.add_handler(CallbackQueryHandler(button_handler))
```

```
app.add_handler(PreCheckoutQueryHandler(precheckout_callback))
```

```
app.add_handler(MessageHandler
```

```
er(filters.SUCCESSFUL_PAYMEN  
T,  
successful_payment_callback)  
)
```

```
app.add_handler(MessageHandl  
er(filters.TEXT &  
~filters.COMMAND,  
text_message_filter))
```

نوسان خودکار قیمت بازار هر ۱
ساعت

```
app.job_queue.run_repeating(  
    hourly_market_job,
```

```
    interval=MARKET_UPDATE_INTER  
VAL_SECONDS,
```

```
    first=MARKET_UPDATE_INTERVAL  
_SECONDS,  
    )
```

```
        logger.info("LIBER bot  
(all-in-one) started.")  
        app.run_polling()
```

```
if __name__ == "__main__":  
    main()
```